Data Storage in a DBMS

Ioana Manolescu
Inria Saclay and Ecole Polytechnique



Plan

- We have seen:
  - Conceptual view of the data: entities, relationships
  - Logical view of the data: relations
  - Query formalisms
- Today we start considering the **concrete implementation** of a Database Management System (DBMS)
  - Hardware for persistent data storage
  - Storing the data from one relation
  - Finding data in one relation
  - Finding data faster: indexes
    - Tree-structured indexes (first part, to be continued)

Why is this important?

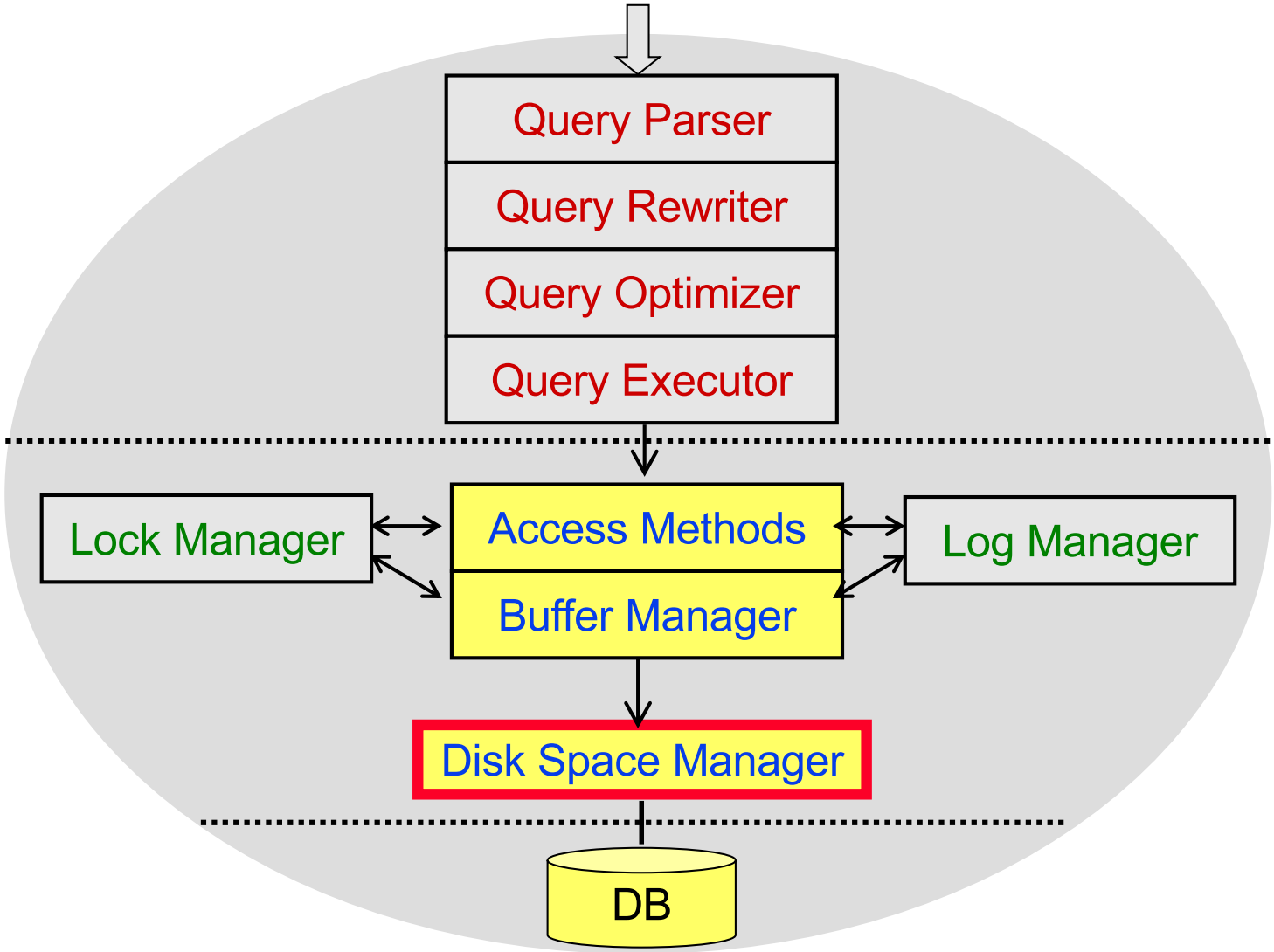
- **Access to data** (traversing it, searching for the interesting tuples etc.) is one of the two main factors determining the time it takes to process a query or update

- The other main factor is the **cost of evaluating operators**, in particular
  - How much it takes to evaluate each join, selection etc.
  - What is the best order in which to evaluate them
  - To be seen later in this course

Disks, Files, Buffers



DBMS Architecture



Outline

- Disks, Disk Space Manager
- Disk-Resident Data Structures
  - Files of records
  - Indexes (next lecture)
- Buffer Manager

Memory Hierarchy

- Main Memory (RAM)
  - Random access, fast, usually volatile
  - Main memory for *currently used data*
- Magnetic Disk (hard disk, HD)
  - Random access, relatively slow, non-volatile
  - Persistent storage for all data in the database.
- Solid State Drive (SSDs)
  - Random access, quick, non-volatile
  - Expensive, shorter lifespan than disk
- Tape
  - Oldest storage medium, some still in use today
  - Very durable, very cheap
  - Sequential scan → very slow!

Programs (including the database server) need the data here!

At any time, data is *stored* (*persisted*) in one or more of these

Memory Hierarchy: Tape

- Oldest storage medium, some still in use today
- Very durable, very cheap
- Sequential scan → very slow
- Useful for data gathered and stored in order to comply with legal obligations, but not needed by the business in normal functioning
- Easier to protect from intrusion
- Low carbon footprint
- "Make space" for mission data on faster stores



https://www.lemonde.fr/sciences/article/2023/01/13/stockage-de-masse-le-retour-de-la-bande-magnetique_6157727_1650684.html

ACTUALITÉS ÉCONOMIE VIDÉOS DÉBATS CULTURE LE GOÛT DU MONDE

SCIENCES

Stockage de masse : le retour de la bande magnétique

La bande magnétique de nos cassettes musicales d'antan a le vent en poupe. Non pas pour écouter des chansons, mais pour la préservation des énormes quantités de données que produisent les sociétés numériques.

Par David Larousserie et Audrey Lagadec

Publié le 13 janvier 2023 à 12h00

Offrir l'article

Article réservé aux abonnés

L'un des leaders du stockage de masse, IBM, commercialise depuis la fin 2022 des cassettes de 18 téraoctets, environ 18 fois plus qu'un disque dur courant d'ordinateur. L'armoire électronique Diamondback gérant plusieurs de ces cassettes promet plus de 27 pétaoctets de stockage (1 péta vaut 1 000 téra) pour une durée garantie de trente ans. Même si l'accès est moins rapide que pour les disques durs, la pérennité est plus longue et la consommation électrique plus faible.

L'entreprise fait valoir aussi l'argument de la sécurité, car ces supports servant de sauvegarde « froide », non directement connectée au cœur des machines, sont protégés naturellement des virus présents sur les systèmes « chauds ».

Et ce n'est pas fini, IBM a trouvé en 2021 de nouveaux matériaux qui augmentent, pour l'instant en laboratoire, de 25 fois la densité d'informations stockables par centimètre carré.

Armoire de stockage Diamondback
L'armoire d'IBM possède 1 584 cassettes à bande magnétique. Elles sont stockées et récupérées à la demande par un système robotisé qui les amène à l'un des 14 lecteurs

2 m
Lecteur de bandes
Système robotisé
Stockage des cassettes
1,2 m
60 cm
Bande magnétique
Bande non magnétique
Support
Couche du fond

Fonctionnement de l'écriture/lecture

Ecriture des données
Une bobine, dans laquelle un courant circule, induit un champ magnétique qui oriente les cristaux selon le pôle Nord ou Sud

Lecture des données
La circulation du courant dans le capteur est affecté par le champ magnétique, ce qui permet de déduire la valeur du champ de chaque zone de la bande

Capteur
Bobine
Sens de déroulement de la bande
Champ magnétique
Bande magnétique
Direction de l'aimantation codant les bits 0 ou 1
1,19 µm
40 à 60 nm
46 nm

Les cristaux
Des agrégats de ferrite de baryum se magnétisent. IBM a trouvé que des ferrites de strontium sont encore plus efficaces et à l'avenir augmenteraient la densité d'informations stockables.
Environ 10 nm

Source : IBM
Infographie : *Le Monde*, Audrey Lagadec

David Larousserie et Audrey Lagadec

Édition du jo
Daté du mardi 31 janvie
Retraites : les clés de l
Fin de vie : le dilemme des juges
Lire le journ
Lire les éditio

Les plus lus
1 Boris Johnso comment Vla « menacé » av guerre en Ukr
2 Guerre en Uk le Kremlin dé menaces de à l'encontre d
3 Les construct Nissan, unis vingt ans, sig divorce

Disks and DBMS Design

- DBMSs have been around since ~ 1970
- From 1970 and still up to today, the main stable storage is provided by disks.
- Major implications on DBMS design
  - READ: transfer data from disk to RAM for **data processing**.
  - WRITE: transfer data (new/modified) from RAM to disk for **persistent storage**.
  - Both are high-cost operations relative to in-memory operations, so must be planned carefully
- Advanced/modern storage:
  - SSD
  - Memory

Basics of Disks

- Unit of storage and retrieval: *disk block* or *page*.
  - A contiguous sequence of bytes.
  - Size is a DBMS parameter, 4KB or 8KB (for PostgreSQL).

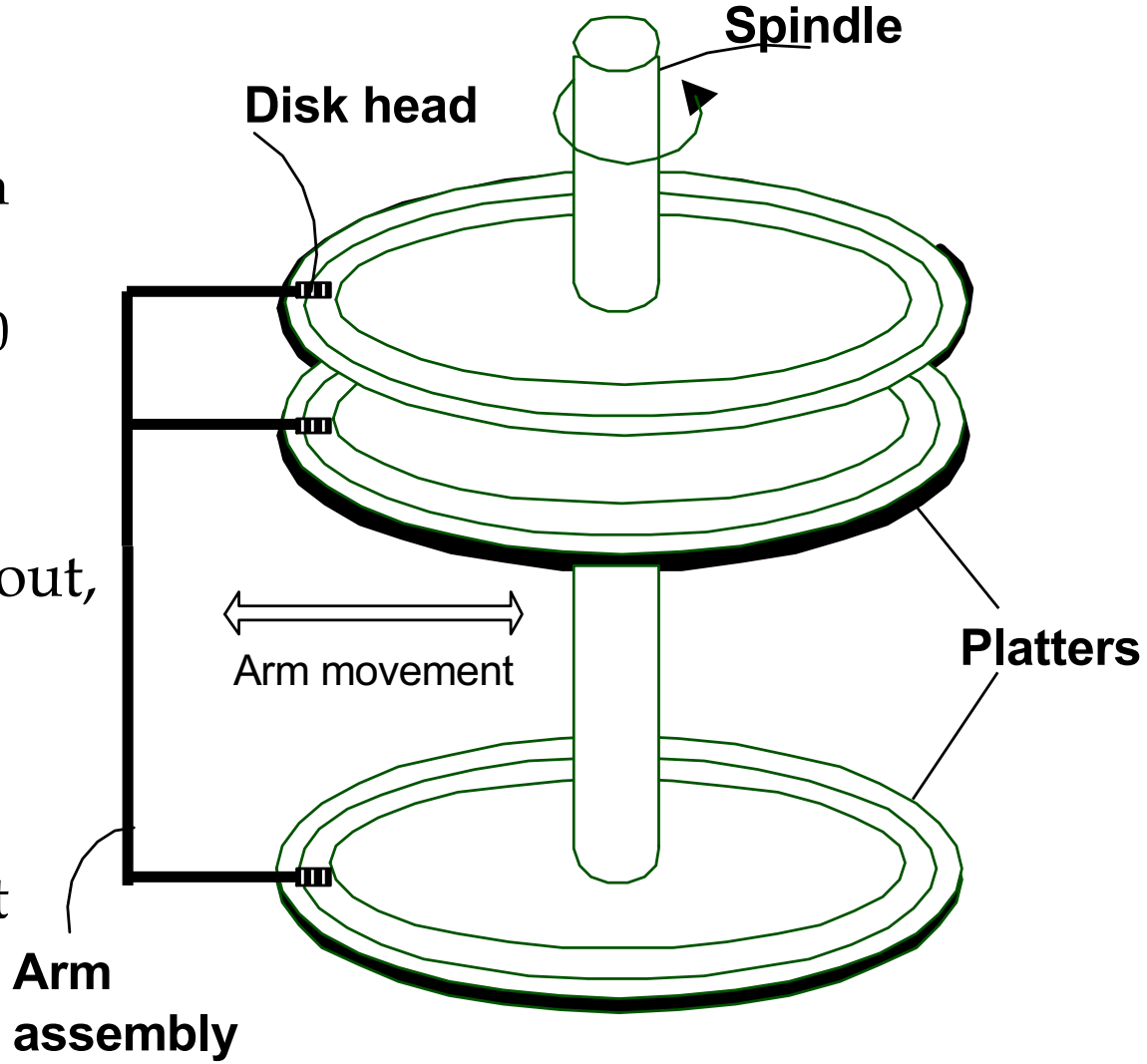
- Unlike RAM, **time to retrieve a page varies**
  - It depends upon the *location of the page on disk*.
  - Relative placement of pages on disk has major impact on disk-based DBMS performance

Components of a Hard Disk

- *Spindle*, *Platters*

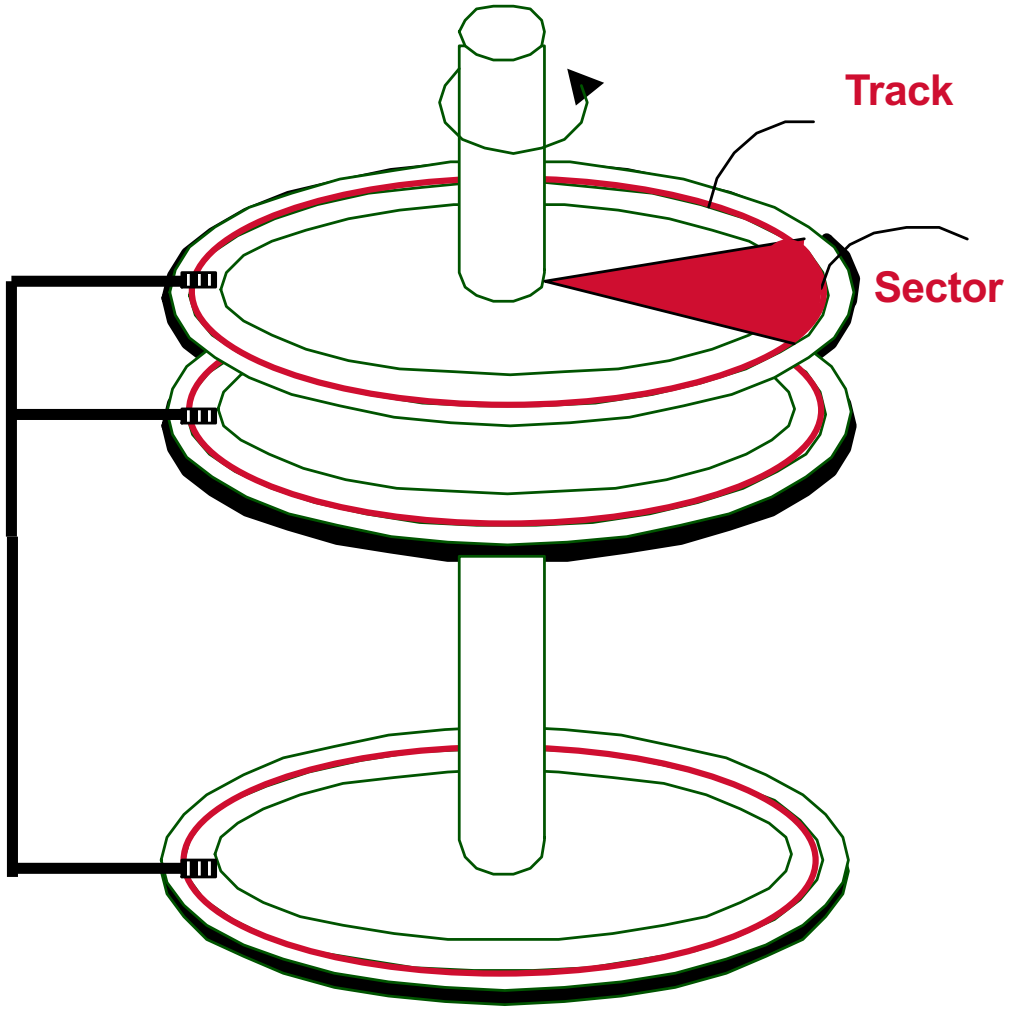
  E.g. spin at 7,200 or 15,000 rpm (revolutions per minute)
  Fast enterprise HDD: 10-15,000

- *Disk heads*, *Arm assembly*
  - Arm assembly moves in or out, e.g., 2-10 ms
  - In the current state of technology, mostly only one head (max 2) reads/writes at any one time.
  - Hard to perfectly align more heads.



Data on Disk

- A platter consists of *tracks*.
  - single-sided platters
  - double-sided platters
- Tracks under heads make a *cylinder* (imaginary!)
- Each track is divided into *sectors* (whose size is fixed).
- *DBMS block* (*page*) *size* is a multiple of *sector size*.



Accessing a Disk Page

- Time to access (read/write) a disk block:
  1. *seek time* (moving arms to position a disk head on a track)
  2. *rotational delay* (waiting for a block to rotate under the head)
  3. *transfer time* (actually moving data to/from disk surface)
- **Seek time and rotational delay dominate**.
  - *seek time*: 2 to 10 msec
  - *rotational delay*: 0 to 10 msec
  - *transfer rate*: < 1msec/page, or 10's-100's megabytes/sec
  - significant density/speed improvement in 2000' thanks to giant magnetoresistance (A. Fert @ UPSud, Grunberg)
- Key to lower I/O cost: reduce seek/rotation delays!
  - Hardware solution: better disk
  - Software solution: make the best use of what you've got.

Arranging Pages on Disk

- The closest that two records can be: **same block**
- The closest two blocks can be is: in the **same track**
  - One track is active at a time and no seek is needed
  - In current disks, a track can be fully read in one revolution
- Otherwise, best is: **same cylinder**
  - The second head can start reading after the first, without seek time
- Otherwise, best is: **adjacent cylinders**

- Records in a *file* should be arranged sequentially on disk (as above), to **minimize seek time and rotational delay**.
  - Scan of the file is a *sequential scan*.

Disk Space Manager

- Lowest layer of DBMS managing space on disk.
- Provides the abstraction of data **page**
  - E.g., "A page of Sailor records"
- Higher levels call it to:
  - allocate/de-allocate a page
  - allocate/de-allocate a sequence of pages
  - read/write a page

- Requests for a sequence of pages are satisfied by *allocating the pages sequentially* on disk!
  - Higher levels don't need to know any details.

Solid State Drives (SSD)

- Increasing share of disks in new computers
- « Solid » = Recent alternative to previous storage whose operation required moving
  - Magnetic tape
  - Disk
- Moving parts are
  - Subject to heating → need to ventilate → size constraints
  - Subject to breaking through mechanical shocks

- SSD store data using electrically-programmable non-volatile (persistent) flash memory
  - Based on **semiconductors**

Solid State Drives (SSD)

- The storage capacity wears out after a certain number of writes
  - 3,000/5,000 cycles
  - A *device controller* ensures writes are spread out over the sectors
- They store data in electrical charges, which slowly leak over time if left without power.
  - Data can be lost after a few years without use →
  - SSDs are not suited for archival purposes.
- Time to read a data unit is about the same as the time to write it

SSD to HDD comparison

	Solid State Drives (SSD)	Hard Disk Drives (HDD)
Maximum capacity	60 TB	14 TB
Principles	Fully electronic	Mechanical + electronic
Start-up time	Basically 0	Few ms
Random access time	**< 0.1 ms**	**2.9 to 12 ms**
Needs ventilation	**No**	**Yes**
Data transfer rate	200 MB/s to 4500 MB/s	200 MB/s
Other	Limited number of writes before failure Performance degrades with use; is hard to predict!	Sensitive to heat, cold, mechanical vibration

Disk and memory evolution

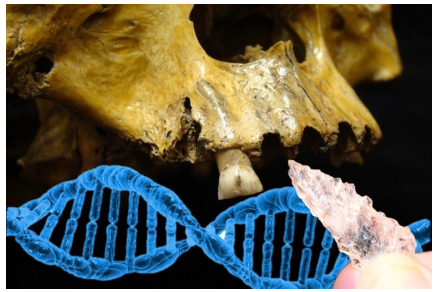
Machine	Cache size	Max main memory	Disk size	Disk transfer rate	Price
1980 server	1 KB	8 MB	80 MB	1.2 MB/s	$250K
2016 server, HD	25 MB $\times 10^4$	126 GB $\times 10^5$	4 TB $\times 10^5$	200 MB/s $\times 10^2$	$6K
2017 laptop, SSD	1 GB $\times 10^6$	16 GB $\times 10^3$	2 TB $\times 10^5$	500 MB/s $\times 10^2$	$3K
2024 laptop, SSD	3.2 GB $\times 10^6$	36 GB $\times 10^3$	2 TB $\times 10^5$	4500 MB/s X 10^3	$3.2K

The *transfer bandwidth per available byte* has been reduced by several orders of magnitude

The *ratio of sequential access speed/random access speed* has increased by one order of magnitude

→ Interest in almost never using random access

Storage on synthetic DNA



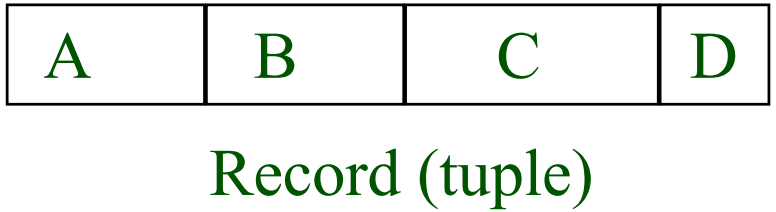
- **DNA can last several centuries**
  - HDD and tape have life times of 5 and 30 years.
- **Extremely dense**: 455 Exabytes (455×10^{18}) / 1 gram;
  - 3.5" hard disk drive: 10 Terabytes (10^{13})/ 600 grams
- **Easy, quick, and cheap to perform in-vitro replication** of DNA
  - Tape and HDD take hours or days for copying Exabyte-sized archives.
- Challenges: oligo size limitations; some "words" are hard to synthesize and read; read/write errors

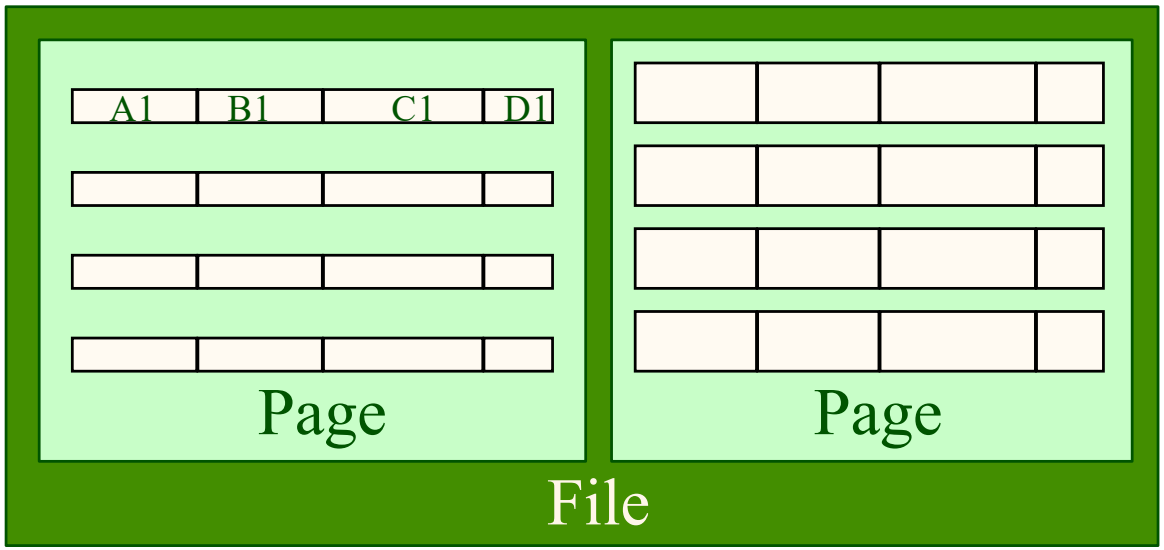
Outline

- Disks, Disk Space Manager
- Disk-Resident Data Structures
  - Files of records – see next (today)
  - Indexes (today)
- Buffer Manager

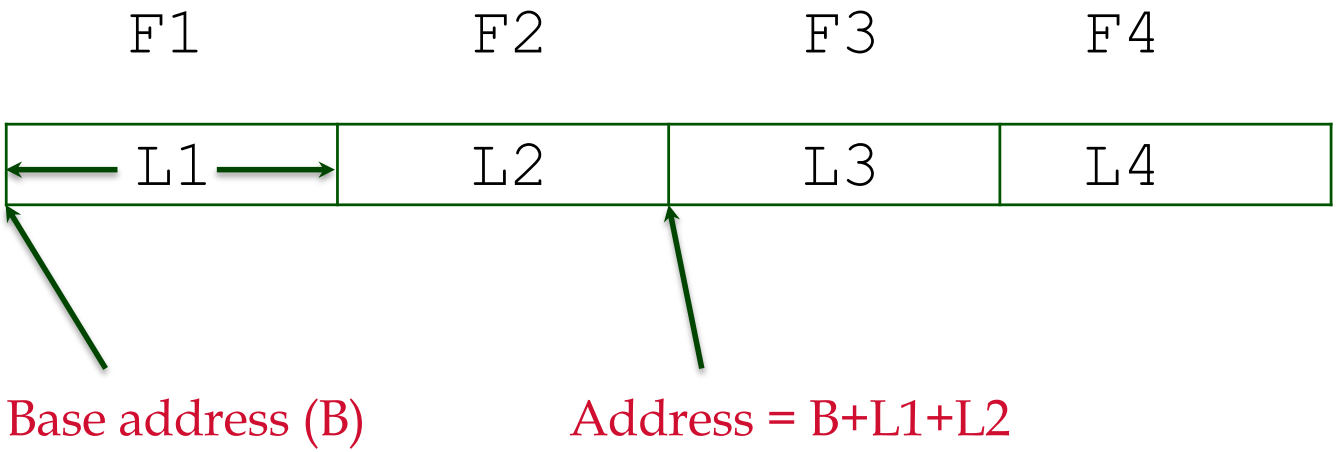
File of Records

- Abstraction of disk-resident data for query processing:
  a file of records residing on multiple pages
  - A number of *fields* are organized in a *record* (tuple)
  - A collection of records are organized in a *page*
  - A collection of pages are organized in a *file*





Record Format: Fixed Length

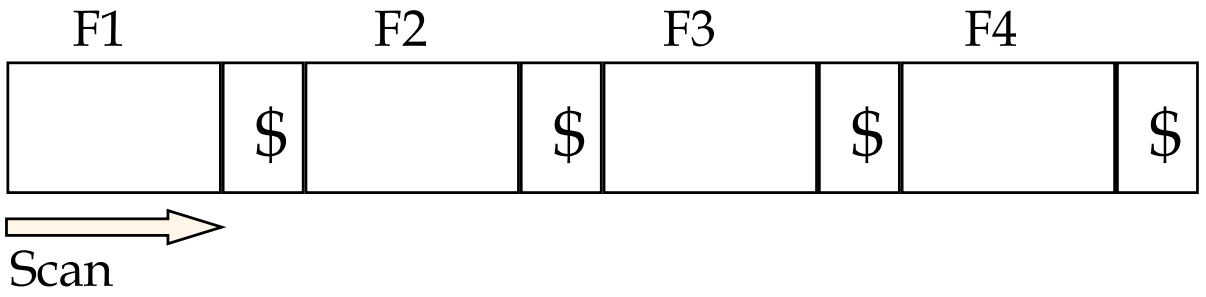


- Record type: the *number of fields* and *type of each field* (defined in the schema), stored in *system catalog.*
- Fixed length record:
  - the number of fields is fixed;
  - each field has a fixed length.
- Fields are stored consecutively in a record.

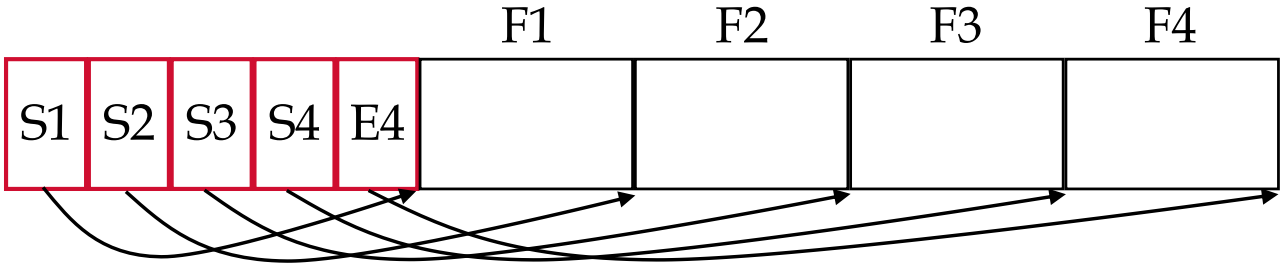
Record Format: Variable Length

- Variable length record: (1) number of fields is fixed, (2) some fields are of variable length

Fields Delimited by Special Symbols



Array of Field Offsets



2nd choice offers direct access to i'th field; but (small) directory overhead.

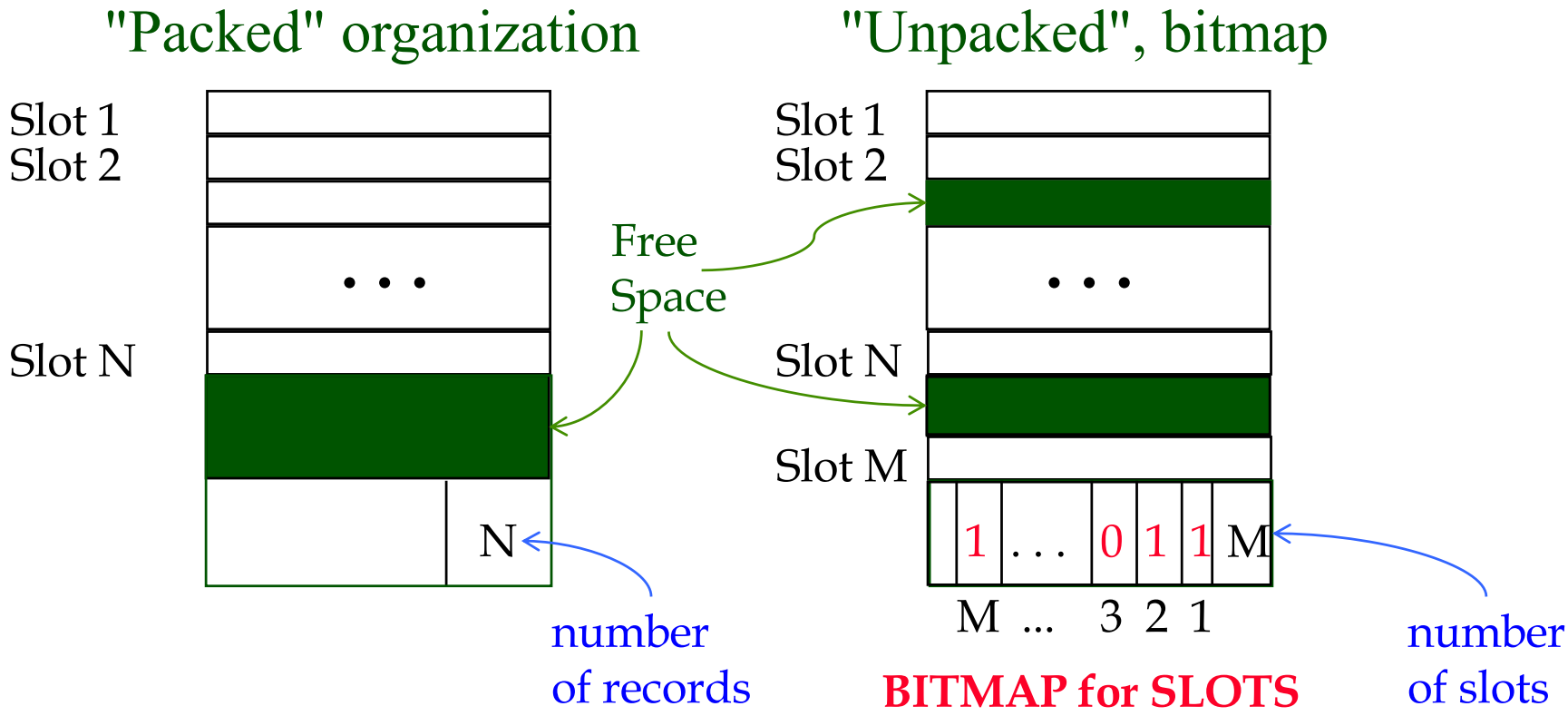
Page Format

- How to store a collection of records on a *page*?

- View a page as a collection of *slots*, one for each record.
- A record is identified by *rid* = <page id, slot #>
  - Record ids (**rids**) are used in indexes (seen next)
  - We should not change rids!

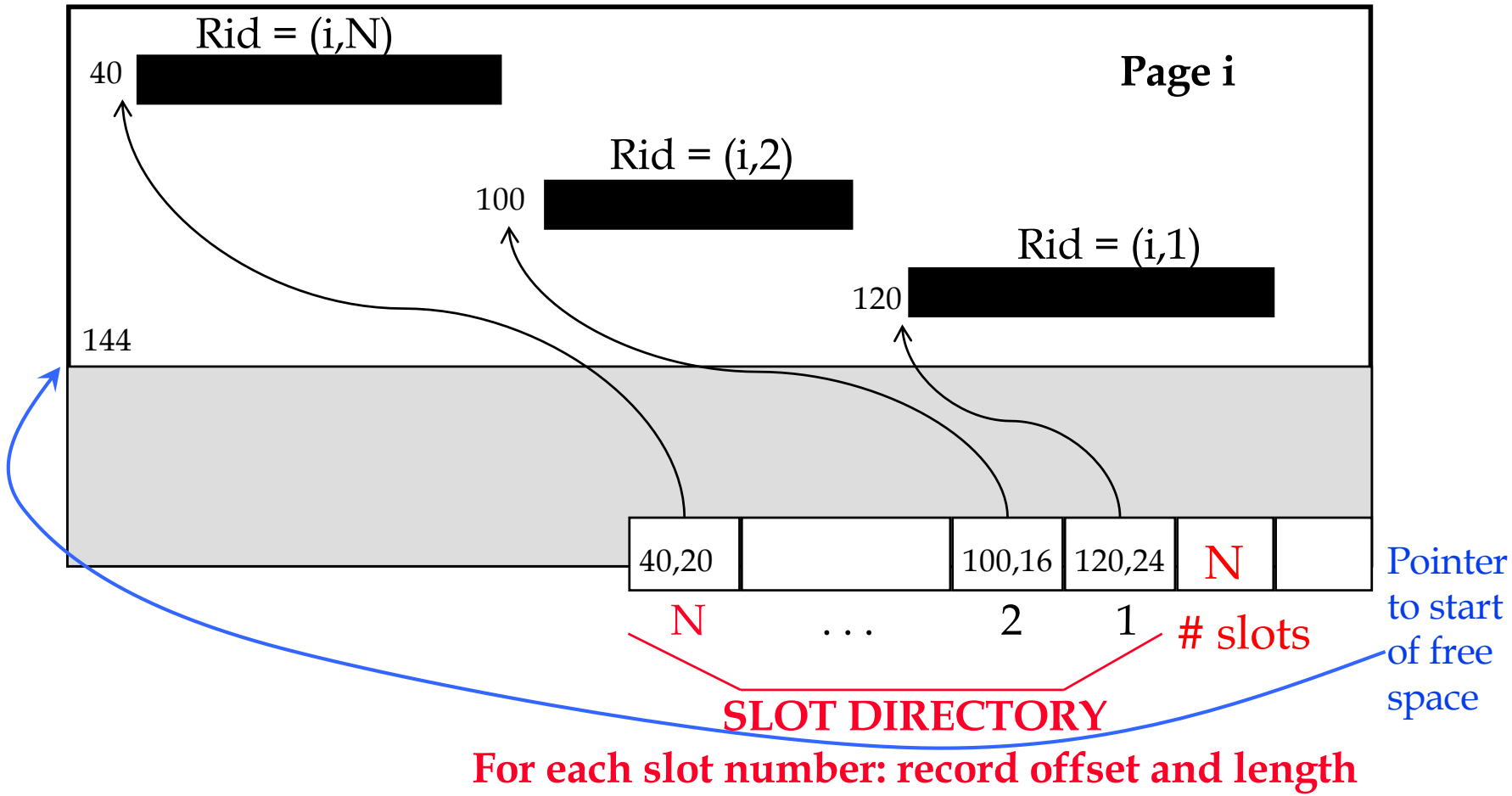
- Different formats depending on whether the records have fixed or varied length
  - create table Sailor(id long, name char(20))
  - create table Sailor(id long, name varchar(20))
  - create table Sailor(id long, name text)

Page Format: Fixed Length Records



"Packed" organization: If we move records for free space management, we would have to change rids.

Page Format: Variable Length Records



Compaction: get all slots whose offset is not -1, sort by start address, move their records up in sorted order.
No change of rids.

Files of Records

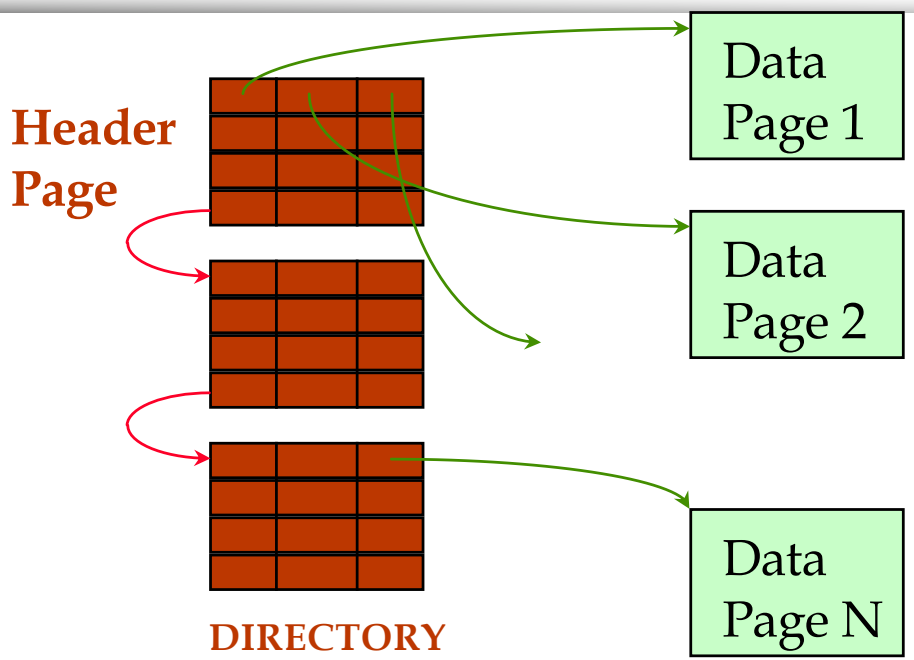
- *File*: **a collection of pages**, each containing a collection of records. Typically, **one file for each relation**.
  - *Updates*: insert/delete/modify records
  - *Index scan*: read a record given a *record id* – more later
  - *Sequential scan*: scan all records (possibly with some conditions on the records to be retrieved)

File according to the OS	File according to the DBMS
Sequence of characters or bytes **Internal structure**: none **Operations**: read bytes (from an offset, for some length)	Collection of **pages** made of **records** **Internal structure**: pages defined by DBMS; records defined by the schema **Operations**: search for records matching conditions, or by rid

Heap (Unordered) Files

- *Heap file*: the "default" form in which the tuples of a relation are stored on disk.
- Contains records in no particular order.
- As a file grows and shrinks, disk pages are allocated and de-allocated.
- To support record-level operations, we must:
  - keep track of the *pages* in a file
  - keep track of *free space* on pages
  - keep track of the *records* on a page
- Typical implementation: page directory

Heap File Using a Page Directory

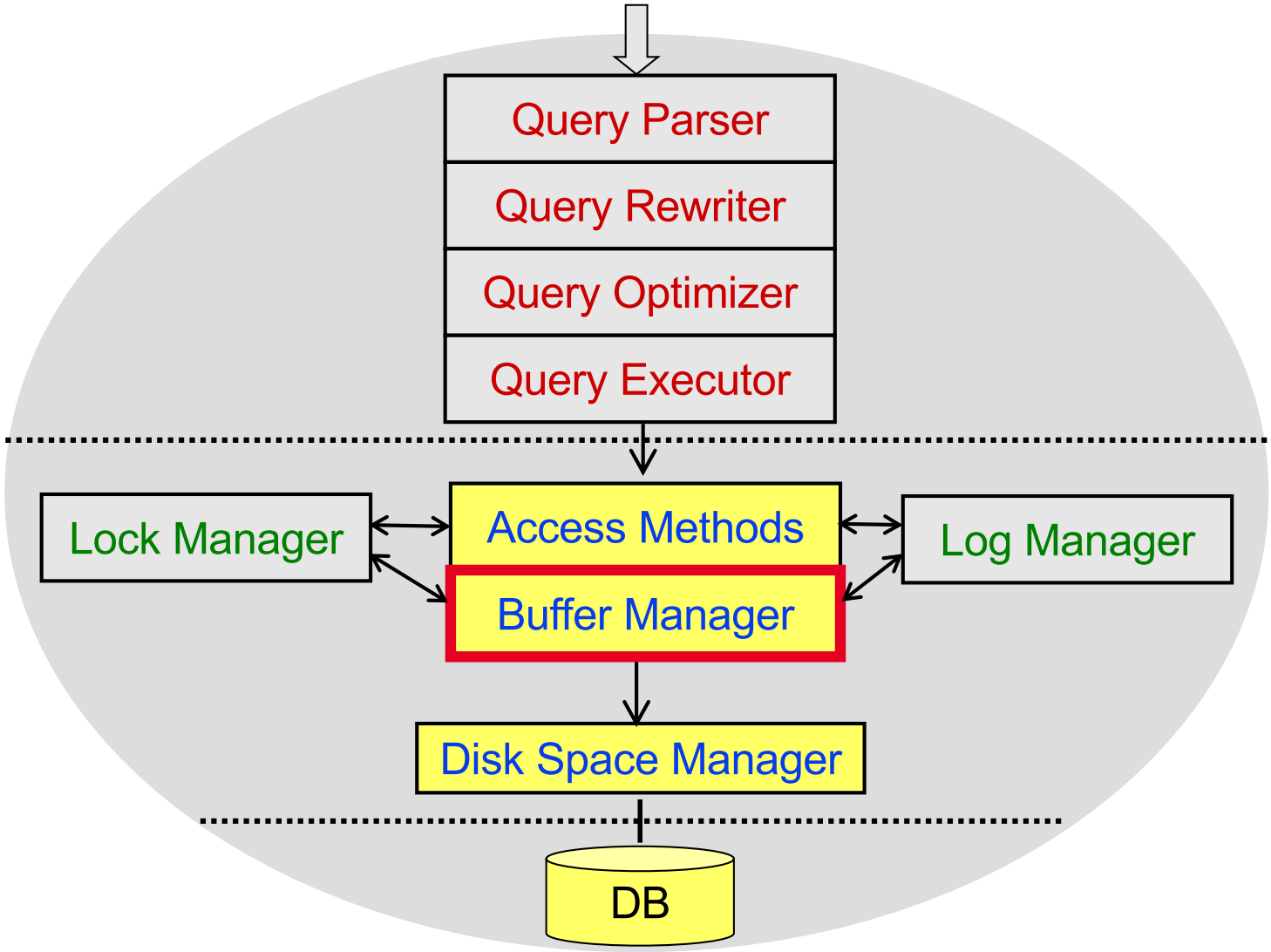


- A directory entry per page: a pointer to the page, #free bytes on the page.
- The directory is a collection of pages; a linked list is one implementation.
  - **Much smaller** than the linked list of all data pages.
- Search for space for insertion: fewer I/Os.

Outline

- Disks, Disk Space Manager
- Disk-Resident Data Structures
  - Files of records
  - Indexes
- (Memory) Buffer Manager

DBMS Architecture



DBMS vs. OS File System

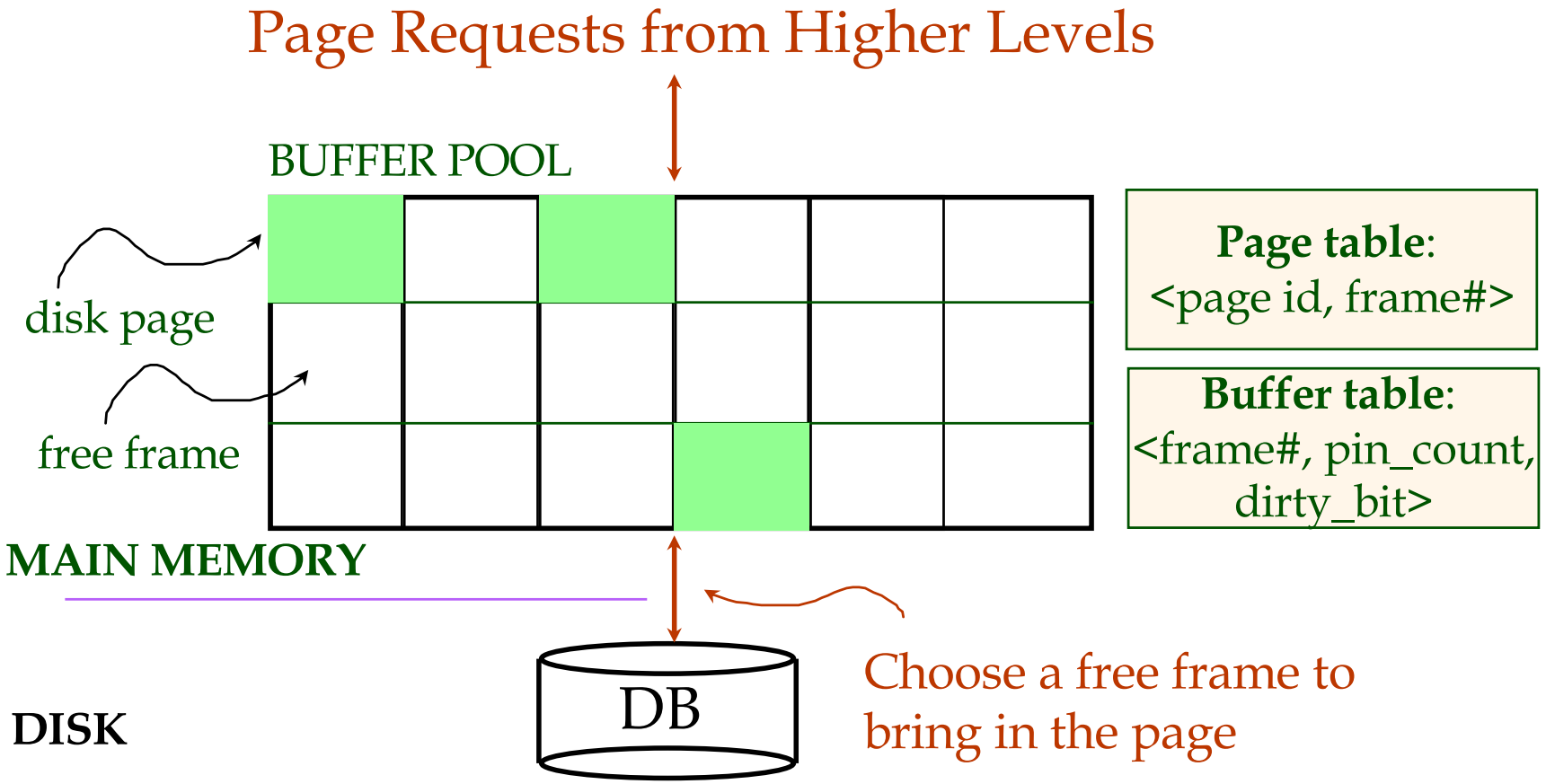
OS does buffer management: why not let OS manage it?

- Differences in OS support: portability issues
- Query-specific data access patterns
  - adjust *replacement policy,* and *pre-fetch* pages based on access patterns in typical DB operations.
- Transaction management
  - *pin* a page in buffer pool: state that one transaction is working on this page (and therefore wants it in memory)
  - *force* a page to disk: propagate back the content of the page to disk (important for implementing concurrency control & recovery)

Buffer Management in a DBMS

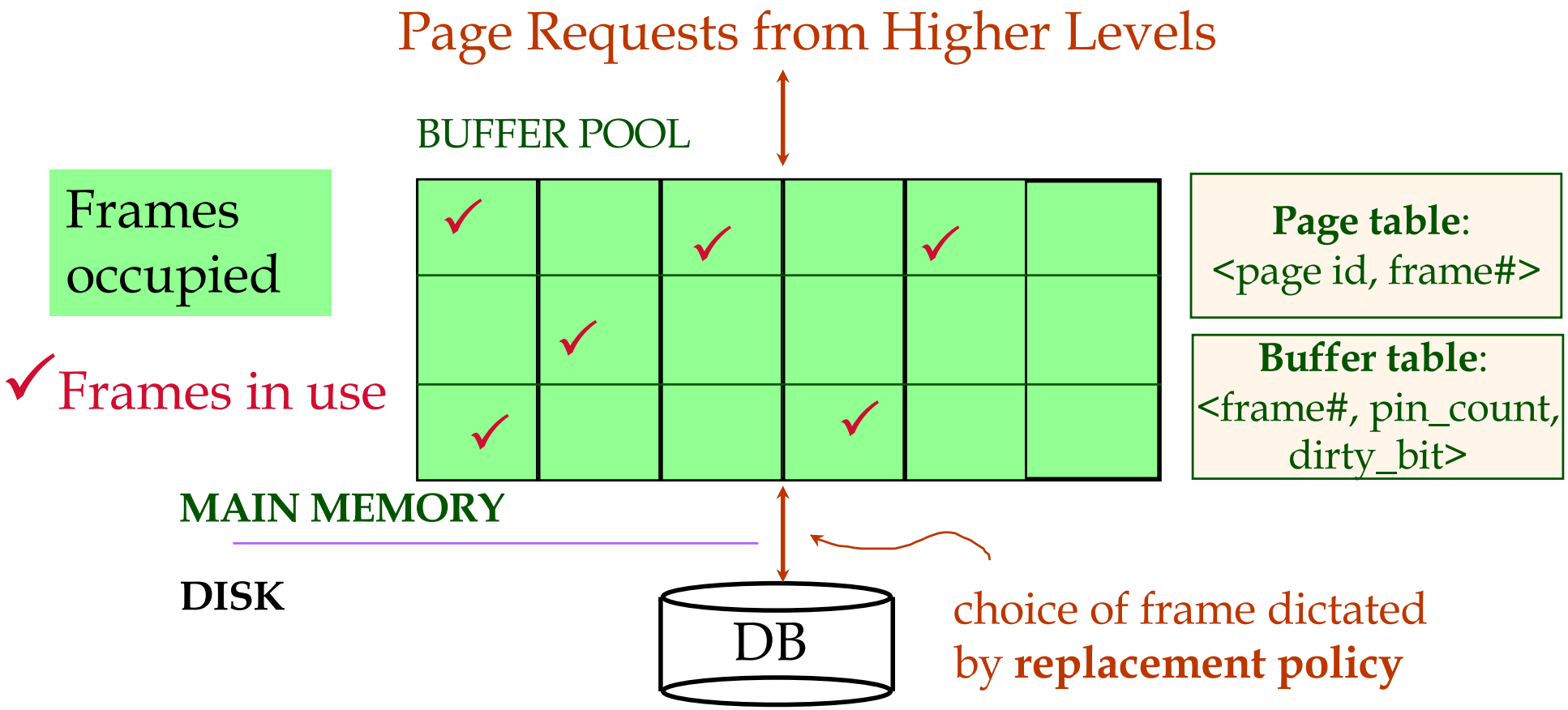
- The buffer manager manages a collection of main-memory pages also called **frames**.
  - A frame can hold the content of a page read from disk.
- The set of all frames is called **buffer pool.**
  - At a certain point in time, some frames are free, while others hold disk pages.
- The mapping from page id to frame # is maintained in a table of <frame#, pageid> pairs.

Buffer Management in a DBMS



- Any queried or updated data goes through the buffer pool

Buffer Management (Contd.)



- When all frames are occupied, evict one frame not in use. Select it using the replacement policy.

When a Page is Requested ...

- If the requested page is not in the buffer pool:
  - Choose a frame for *replacement*
  - If that frame is *dirty*, write it to disk
  - Read the requested page into the chosen frame
- *Pin* the page and return its address to the caller.

☛ If requests can be predicted (e.g., sequential scans) pages can be pre-fetched several pages at a time

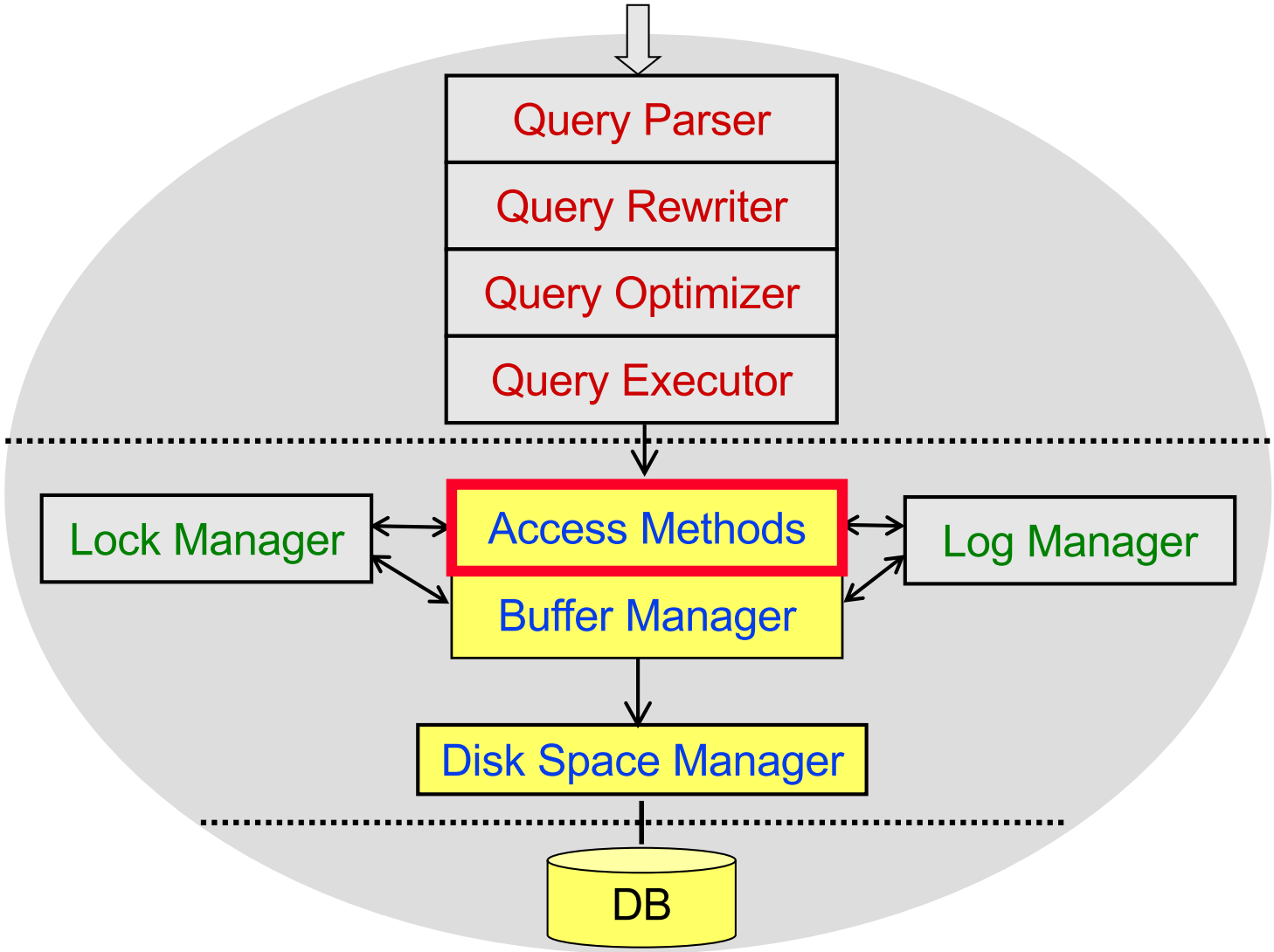
More on Buffer Management

- Requestor of a page must *unpin* it, and indicate if the page has been modified:
  - *dirty bit* is used for this.
- A page in the pool may be requested many times.
  - a *pin count* is used.
  - a page is a **candidate for replacement** if *pin count* = 0.

Buffer Replacement Policies

- A frame is chosen for replacement by a *replacement policy*:
  - **Least-recently-used** (LRU): *based on time of last reference*
  - **Most-recently-used** (MRU): *idem*
  - **First-in-first-out** (FIFO): *based on time of reading the page in*
  - **Last-in-first-out** (LIFO): *idem*
  - **LRU2**: *based on time of 2nd last reference* (general: LRUk)
    - Given a page, the **backward-k** distance of a page is the distance backward to the k-th most recent reference to the page (or infinity if the page has not been referred to k times)
    - Evict the page maximizing backward-k
    - Exploits *frequency of page use*, which LRU ignores

DBMS Architecture

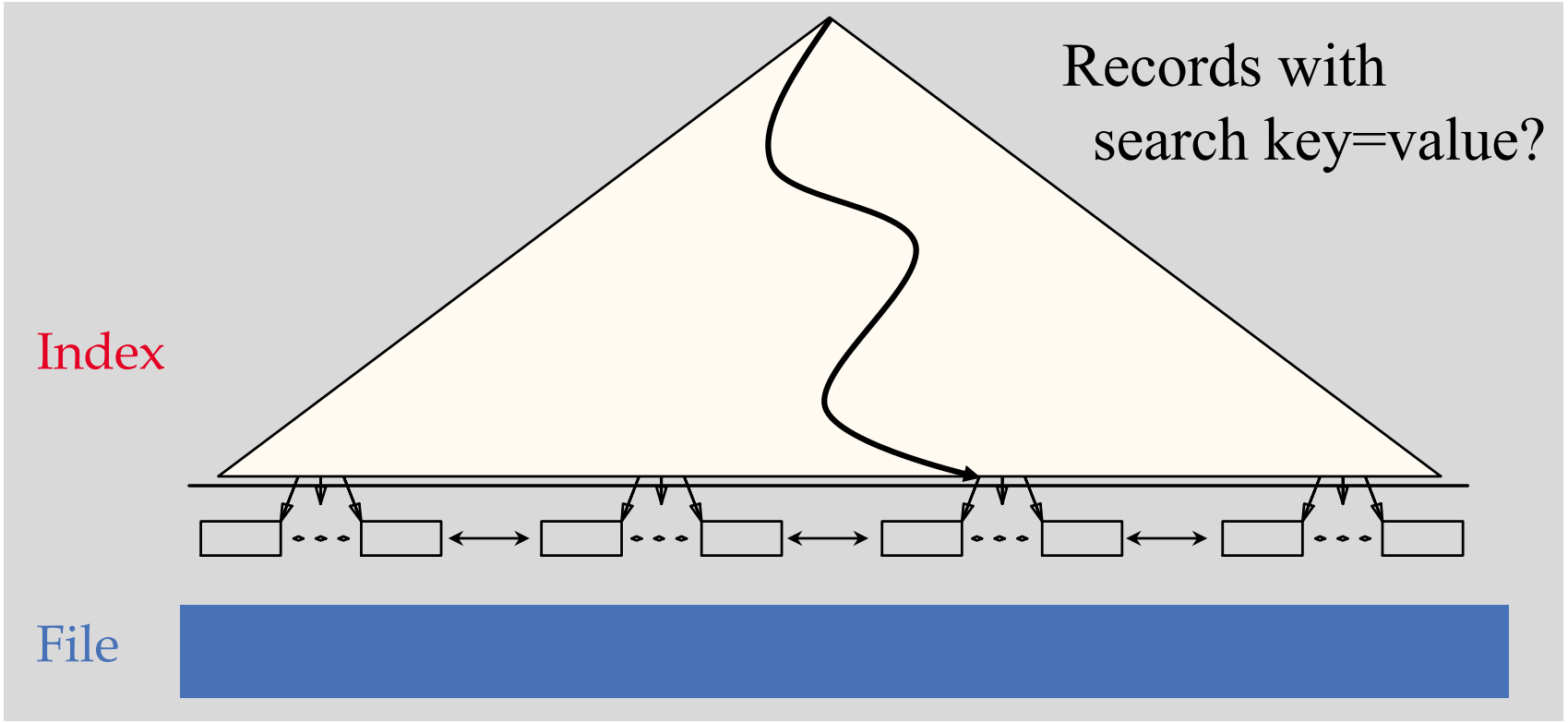


Access Methods

- **Access methods** are routines that manage disk-based data structures.
  - Read, write, search, delete
- Two main such structures:
  - *File of records*:
    - Abstraction of external storage for query processing
    - *Record id* (*rid*) is sufficient to physically locate a record
  - *Indexes*:
    - Auxiliary data structures
    - Given a value in the *index search key* field(s), find the record ids of records with this value

Indexes

- An *index* speeds up access to data *through a selection on the search key* of the index.
  - Indicates where to find such data: record ID(s)
  - Any subset of a relation's attributes can be the index search key.
  - The index search key may or may not be a primary key for the relation



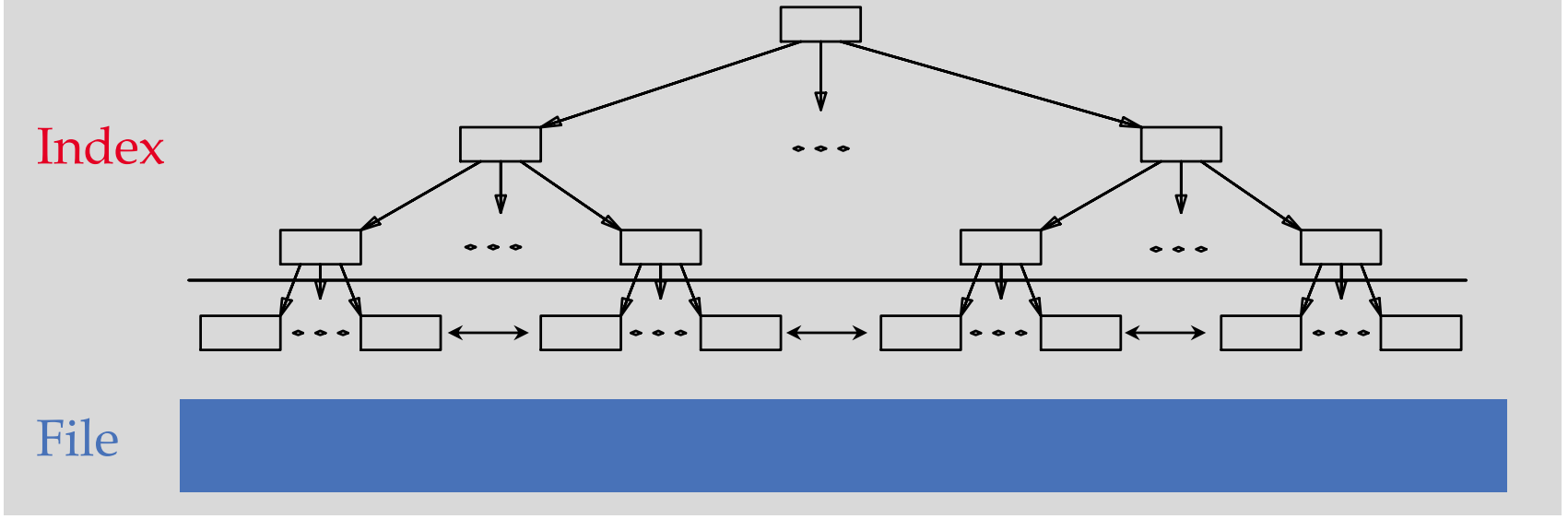
Indexes

- An *index* speeds up access to data *through a selection on the search key* of the index.
  - Indicates where to find such data: record ID(s)
  - Any subset of a relation's attributes can be the index search key.
  - The index search key may or may not be a key for the relation

Records having search key=value?

Particular case of index:

- B+ tree (similar to a search tree)



The details of indexed access

- Given a certain search condition, the index allows finding the *addresses* (page, record #) *of the records* we search for
  - E.g., (p1, r1), (p1, r2), (p3, r3), (p3, r6), (p3, r201)
  - The minimum access unit is a *page* → Read p1 once and p3 once
  - This is called **Bitmap Heap Scan** in Postgres

  (1 bit for every page: should we read this page?)

Sample index configuration

```
ioana.manolescu=# \c test
Vous êtes maintenant connecté à la base de données « test » en tant qu'utilisateur « ioana.manolescu ».
test=#  create table sailors(id int primary key, name varchar(50), address varchar(200));
CREATE TABLE
test=# \d sailors;
                        Table « public.sailors »
 Colonne |          Type          | Collationnement | NULL-able | Par défaut
---------+------------------------+-----------------+-----------+------------
 id      | integer                |                 | not null  |
 name    | character varying(50)  |                 |           |
 address | character varying(200) |                 |           |
Index :
    "sailors_pkey" PRIMARY KEY, btree (id)

test=# create index sname on sailors(name);
CREATE INDEX
test=# \d sailors;
                        Table « public.sailors »
 Colonne |          Type          | Collationnement | NULL-able | Par défaut
---------+------------------------+-----------------+-----------+------------
 id      | integer                |                 | not null  |
 name    | character varying(50)  |                 |           |
 address | character varying(200) |                 |           |
Index :
    "sailors_pkey" PRIMARY KEY, btree (id)
    "sname" btree (name)
```

\d: describe

B+tree index created automatically on PK

Second index explicitly created by the user

When is an index used?

- Exact versus approximate search

EXPLAIN: asks for the optimizer's choice of how to evaluate

```
test=# explain select * from sailors where id=10;
                                QUERY PLAN
-------------------------------------------------------------------------
 Index Scan using sailors_pkey on sailors  (cost=0.14..8.16 rows=1 width=540)
   Index Cond: (id = 10)
(2 lignes)

test=# explain select * from sailors where name='Alice';
                              QUERY PLAN
-----------------------------------------------------------------------
 Index Scan using sname on sailors  (cost=0.14..8.16 rows=1 width=540)
   Index Cond: ((name)::text = 'Alice'::text)
(2 lignes)

test=# explain select * from sailors where name like '%li%';
                         QUERY PLAN
-------------------------------------------------------------
 Seq Scan on sailors  (cost=0.00..11.75 rows=28 width=540)
   Filter: ((name)::text ~~ '%li%'::text)
(2 lignes)
```

Tree-Structured Indexes



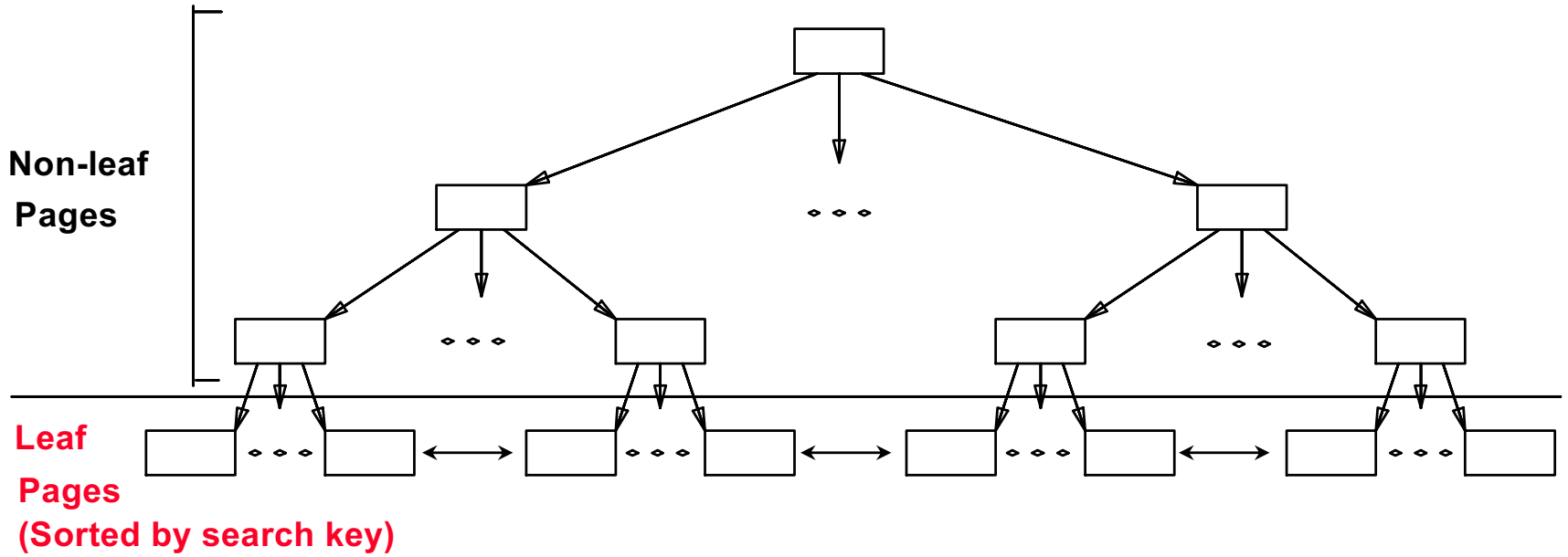
Slides Courtesy of R. Ramakrishnan, J. Gehrke, Y. Diao

Tree-structured indexes

- Index concepts
- ISAM index: static structure
- B+ Tree index: dynamic balanced structure
  - Dynamic construction
  - Bulkloading

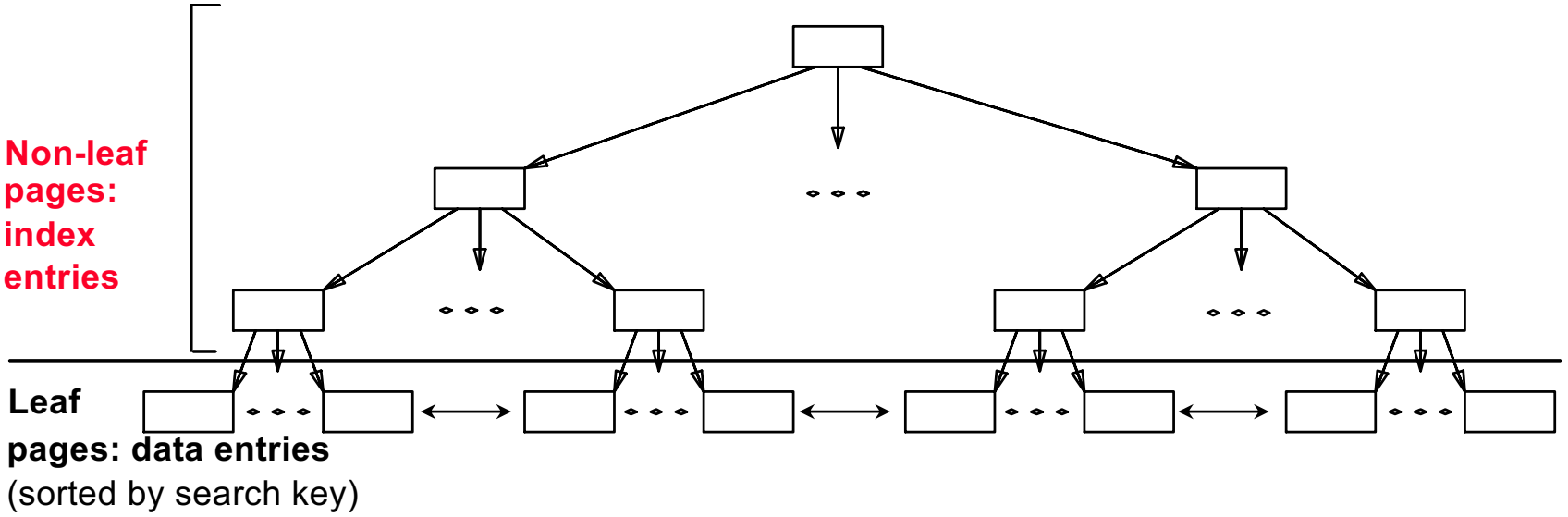
Index concepts

- Illustration on tree indexes; other indexes exist.

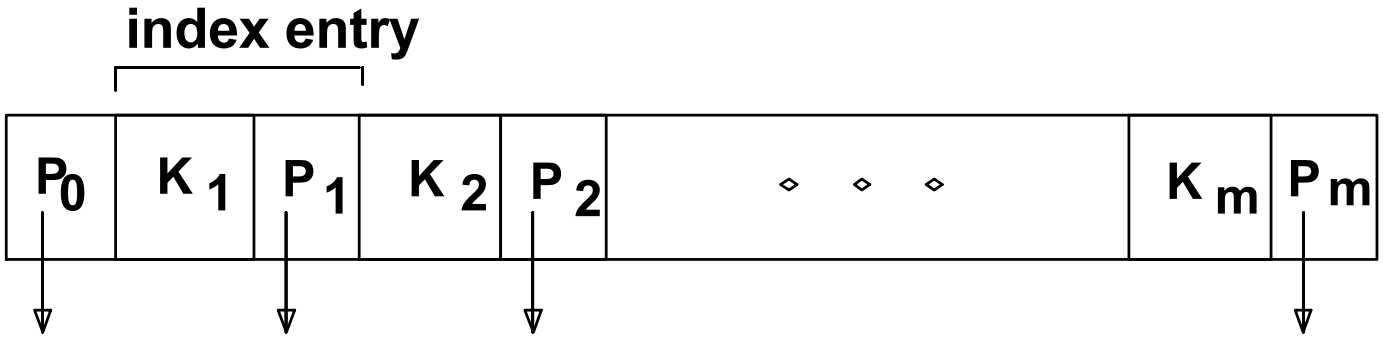


- Leaf pages contain *data entries*:
  - Data entries are *sorted* by the search key value
  - Leaf pages are chained using prev & next pointers

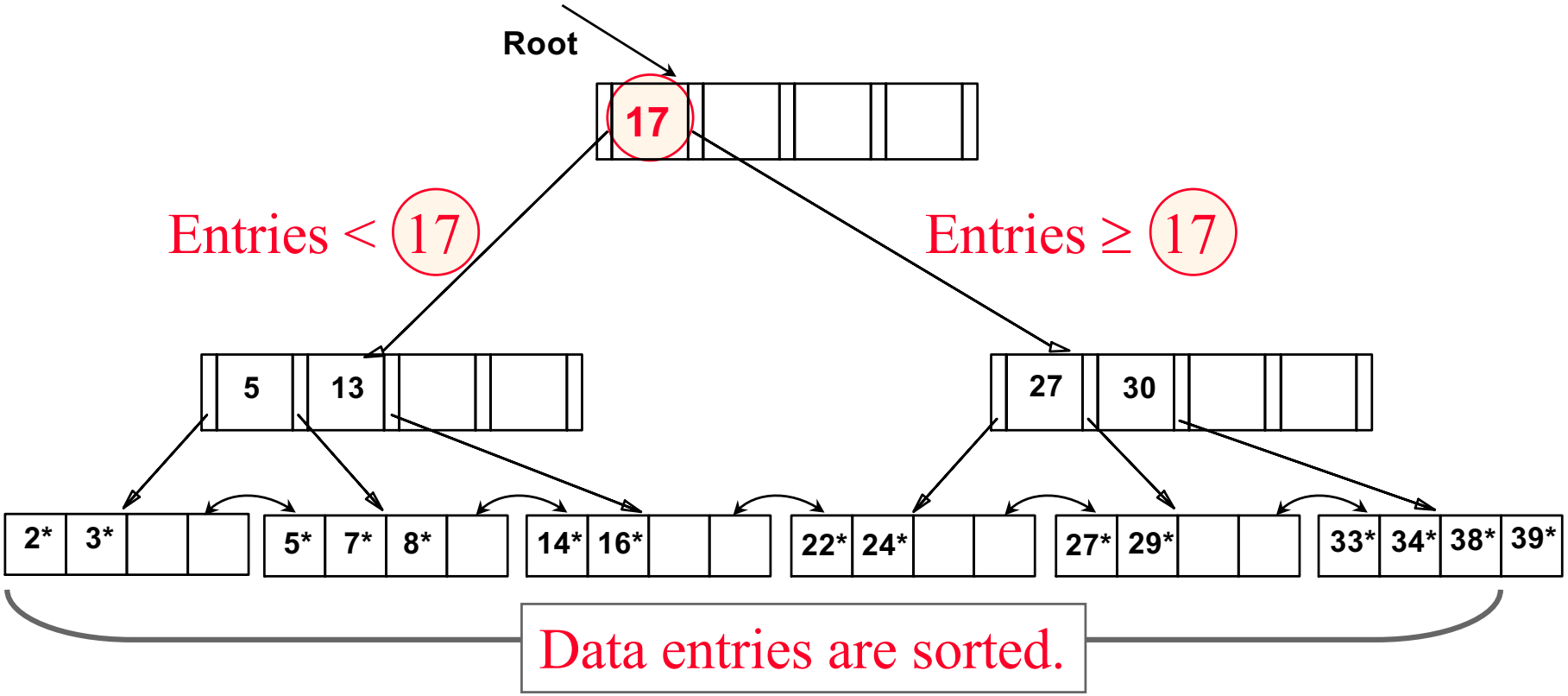
Index concepts



- Non-leaf pages have *index entries*
- Created from data entries, used **only** to direct searches.

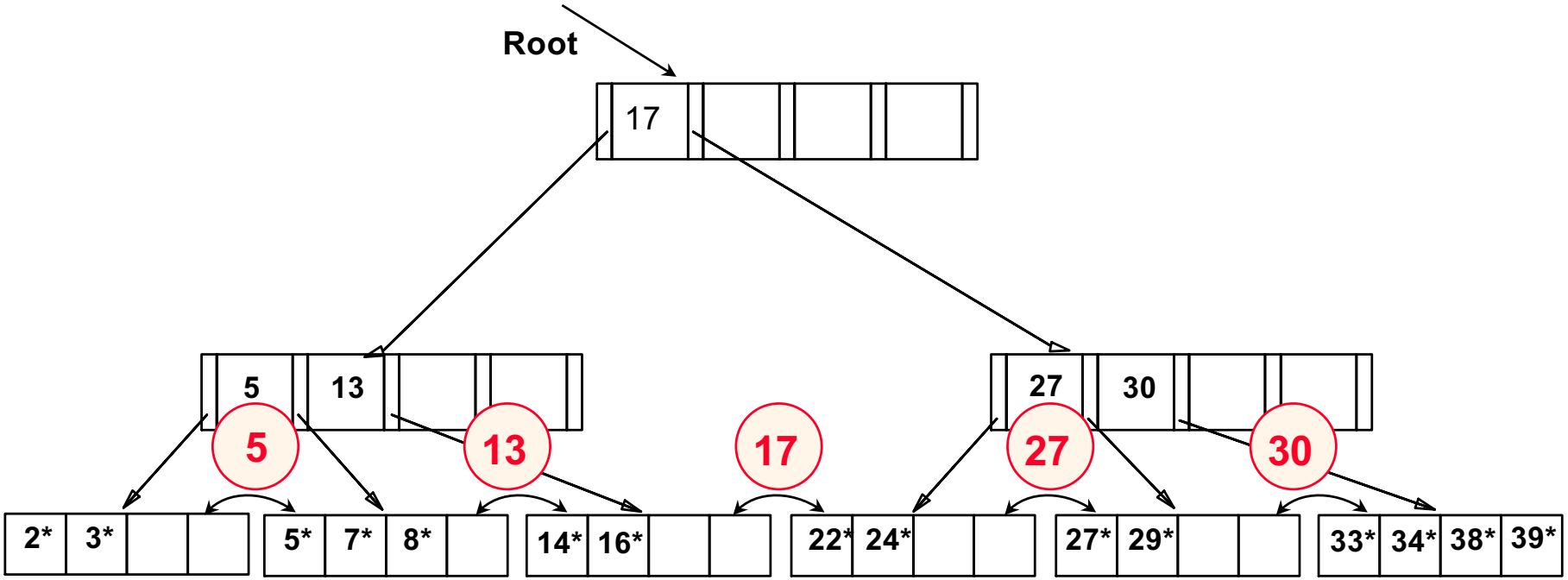


Sample index: B+ tree



- *Equality selection*: find 28*? 29*?
- *Range selection*: find all > 15* and < 30*
- *Insert/delete*: Find the right leaf, then modify it (to be seen)

Sample index: B+ tree



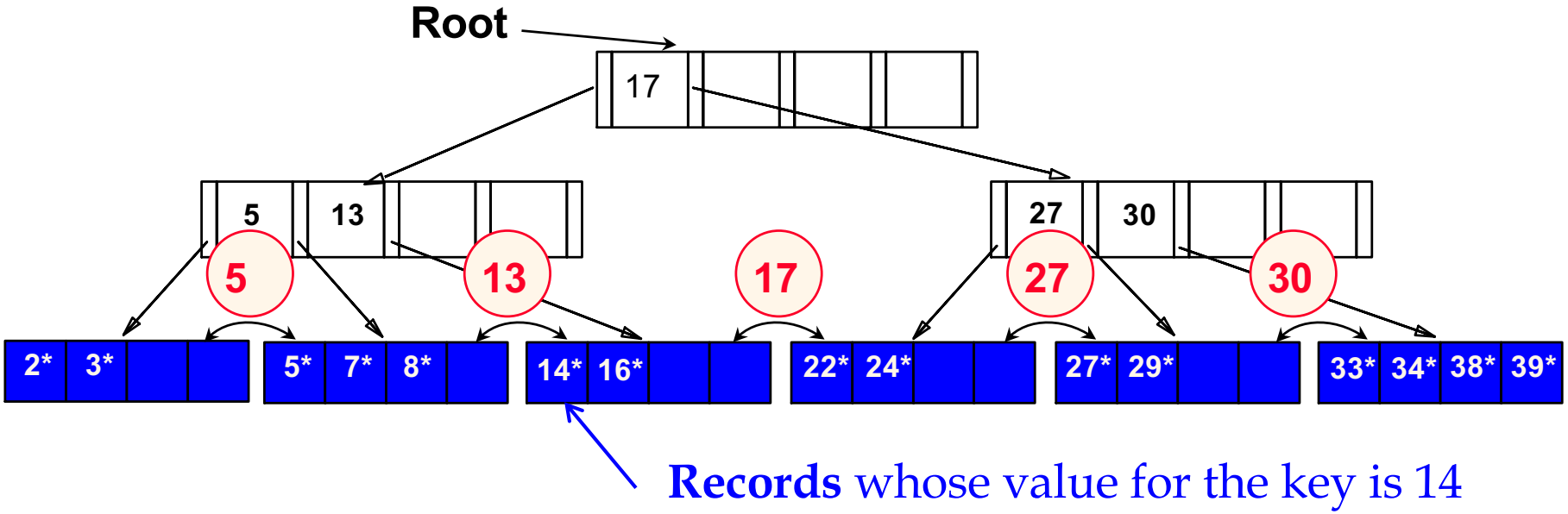
*Alternatives for Data Entry **k*** in Indexes*

- In a data entry k*, we can store:
  - Alternative 1: <**k**, data record with search key value **k**>
  - Alternative 2: <**k**, rid of a record with search key value **k**>
  - Alternative 3: <**k**, list of rids of records with search key **k**>

- Choice of an *alternative for data entries* is orthogonal to the *indexing technique* used.
  - Indexing techniques: B+ tree, hashing, …

Alternative 1 for Data Entries

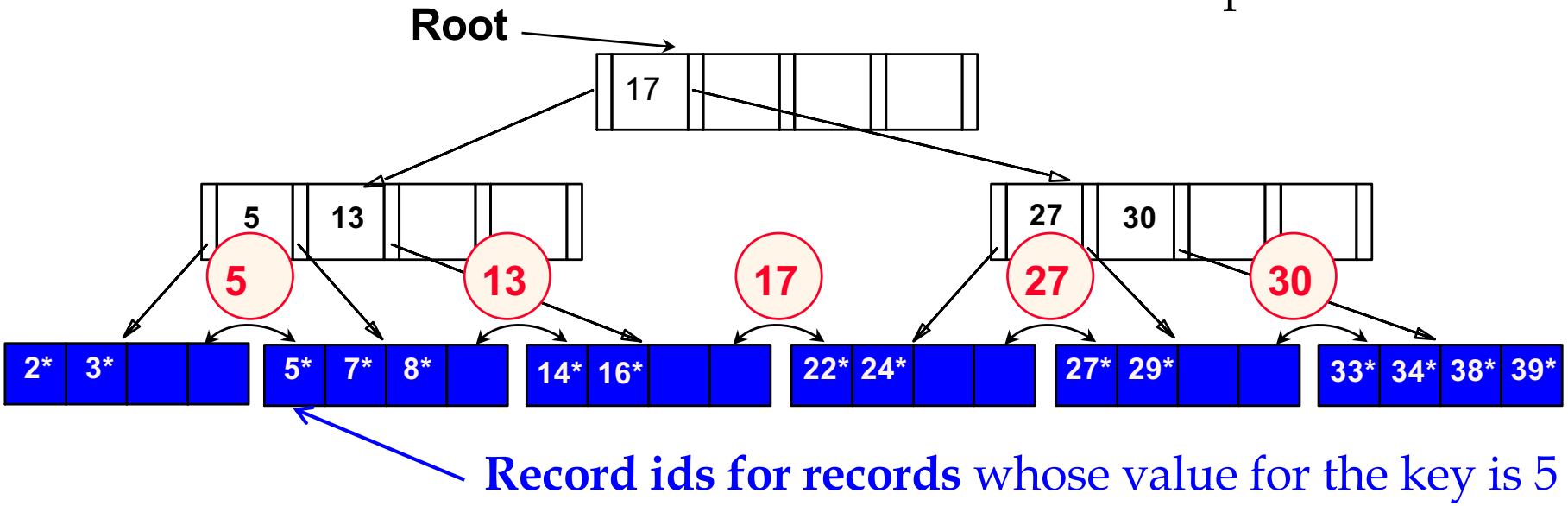
- Alternative 1:
  - Data records are physically stored in leaf pages.



  - At most one index can use Alternative 1.
  - If data records are large, the number of leaf pages is high.
    - The non-leaf part of the index (used to direct searches) can also be large, hence slowing down search.

Alternatives 2, 3 for Data Entries

- Alternatives 2 and 3:
  - <k, rid(s)>: store rid, not the record.
  - Data entries are typically much smaller than data records.
    - Index < data file. Index search structure is compact.



  - Alternative 3: concatenate entries for a given key value
    - Leads to variable sized data entries, even if search keys are of fixed length.

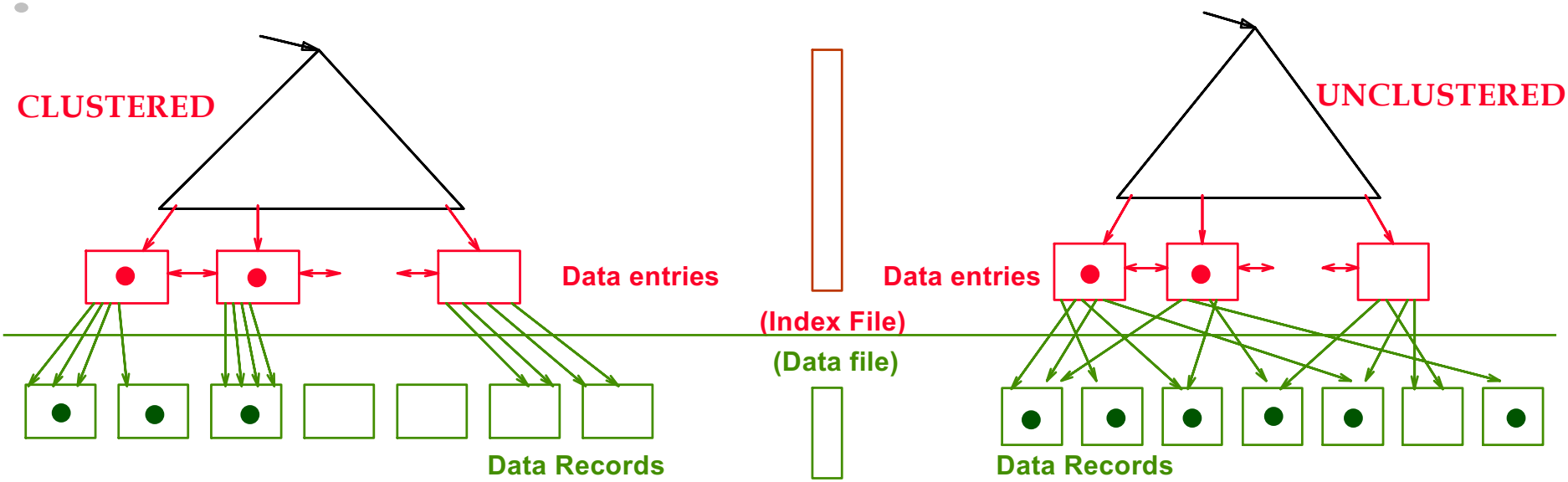
Index Classification (Contd.)

- *Clustered* index: order of data records in a file is the same as or 'close to' order of (sorted) data entries in the index.
  - The data file is (almost) sorted by the index search key.
  - A data file can have at most one clustered index.

  - Alternative 1 with a tree index is always clustered.
  - Alternatives 2 and 3 are clustered only if data records are sorted on the search key field.

- *Unclustered* index: otherwise
  - Multiple unclustered indexes on a data file.

Clustered vs. Unclustered Indexes



- Retrieve a **range of data records** matching a search key value:
  - Clustered index: fast, with one or a few I/Os.
  - Unclustered index: can be slow, reading many pages of data records.
    - 1 I/O per data record in the worst case!



Dense vs. Sparse Indexes

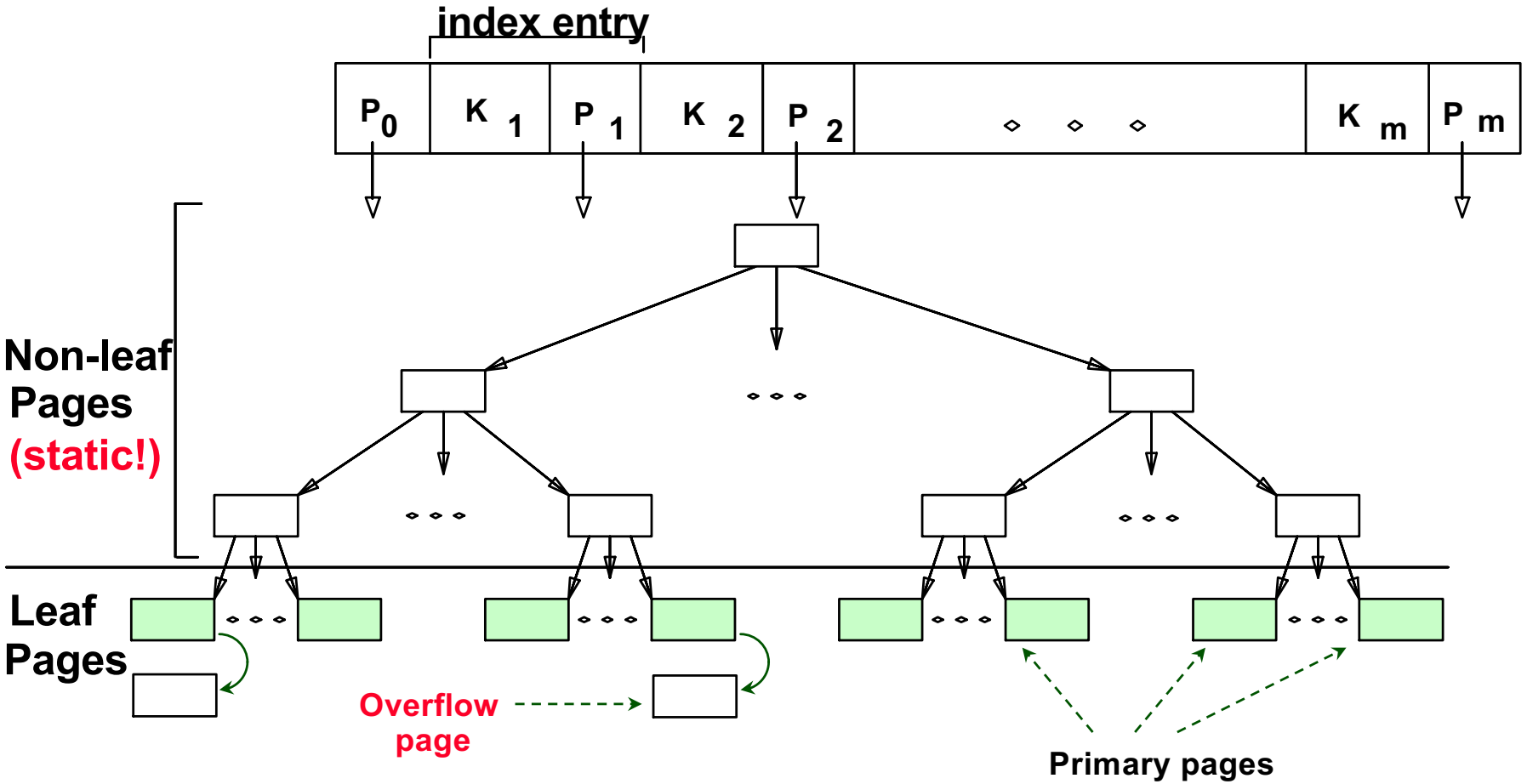
- *Dense index*: contains (at least) one data entry for *every search key value* that appears in a record in the indexed file
  - Alternatives 1, 2, 3

- *Sparse index*: contains one entry for *each page* of records in the indexed file.
  - Sparse index has to be clustered!
  - Typically used for a clustered index using Alternative 2.
  - When the search key is a string, may save a lot of space in non-leaf nodes

Tree-structured indexes

- Index concepts
- ISAM index: static structure
- B+ Tree index: dynamic balanced structure
  - Dynamic construction
  - Bulkloading

ISAM: indexed sequential access method



- Non-leaf part directs searches to the data records; static once built
- **Leaf pages contain sorted data records (e.g., Alt 1 index).**
- Inserts/deletes: use overflow pages, bad for frequent inserts.

Comments on ISAM

- Problems:
  - *Long overflow chains* after many inserts, high I/O cost (# pages accessed) for retrieval.
  - Unsuitable for frequent updates.
- Advantages:
  - Simple when updates are rare.
  - Leaf pages are allocated in sequence, *sequential I/O*.
  - Non-leaf pages are static; for *concurrent access*, no need to lock non-leaf pages!
- To preserve good performance even when updates are frequent, use *B+trees*

Tree-structured indexes

- Index concepts
- ISAM index: static structure
- B+ Tree index: dynamic balanced structure
  - Dynamic construction
  - Bulkloading

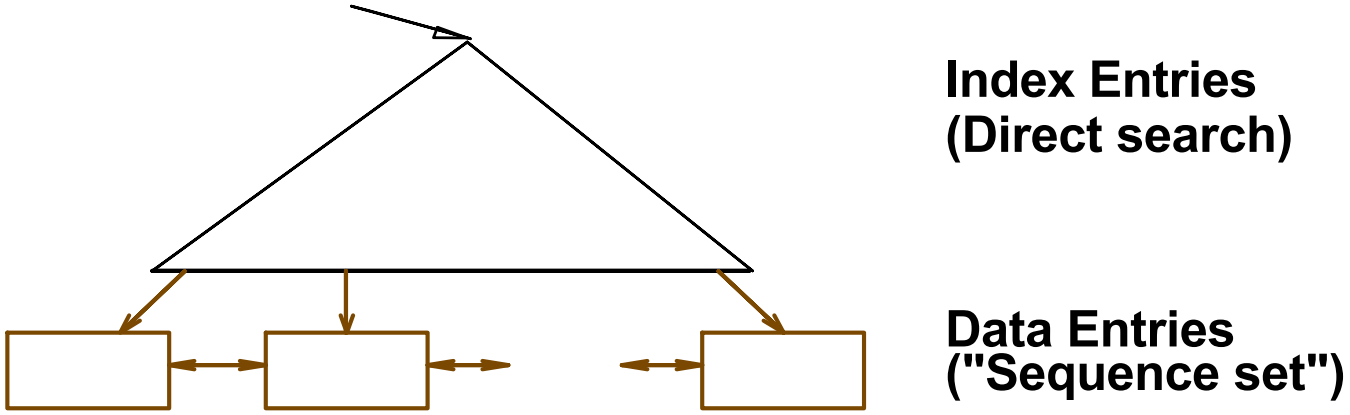
B+ Tree: Most Widely Used Index

- Height-balanced given arbitrary inserts/deletes.
  - *Fanout*: number of child pointers of a non-leaf node

    F = avg. fanout
  - *Height*: N = num. of leaf pages

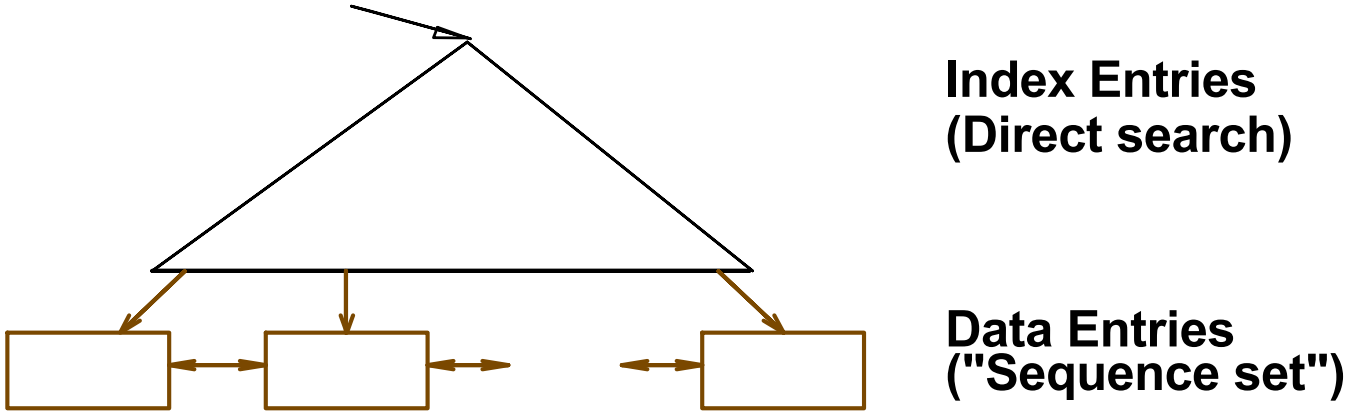
    $H = \text{Log}_F N$

    (Root: level 0, …, Leaf: level H)



B+ Tree: Most Widely Used Index

- Minimum 50% occupancy (except for the root).
  - *Order* of the tree (**n**): max num. of keys in a node.
    - Can be computed using the page size, key size, pointer size.
  - Each non-root node is at least half full, containing at least $\lfloor n/2 \rfloor$ and at most n entries.
  - Root node can have 1 to n entries.

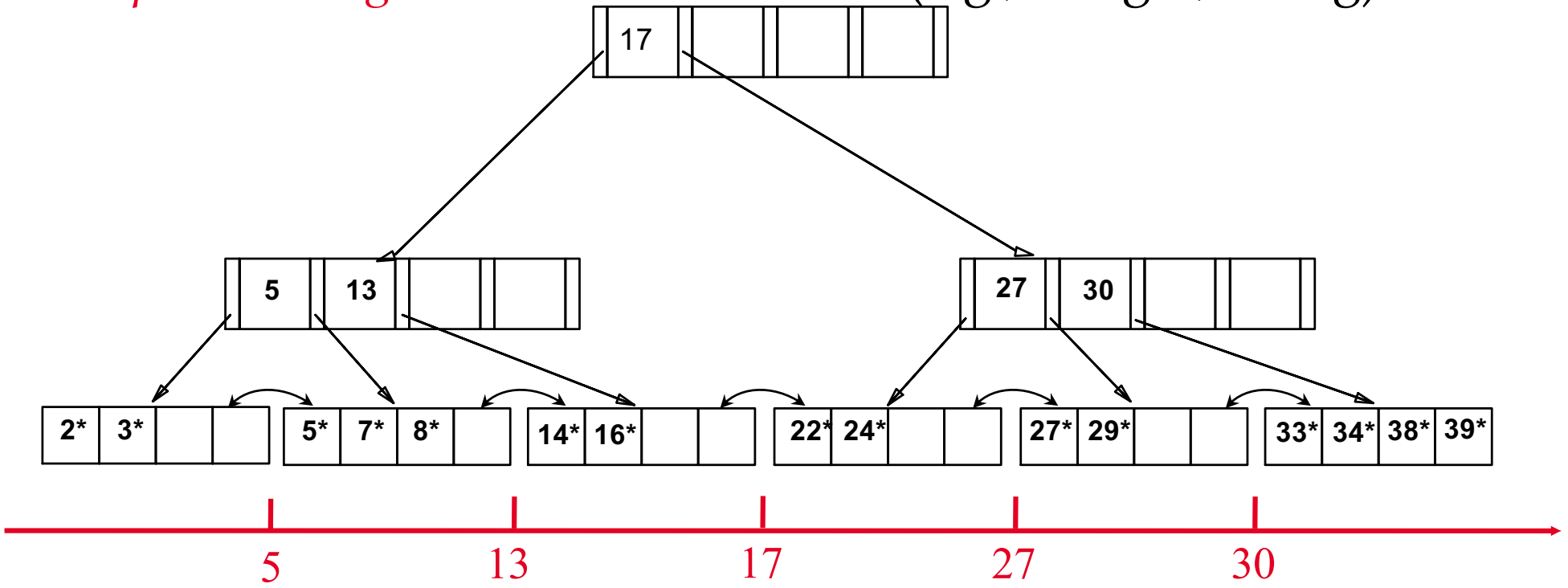


B+ Trees in Practice

- Typical order is 200. Typical fill-factor (occupancy) is 67%.
  - Average fanout = 133
  - Level 0=1 page; Level 1=133 pages; Level 2=133^2 pages...
- Typical capacities:
  - Height 3: 133^3 = 2,352,637 records
  - Height 4: 133^4 = 312,900,700 records
- Can often hold top levels in buffer pool:
  - Level 0 = 1 page = 8 KBytes
  - Level 1 = 133 pages = 1 MByte
  - Level 2 = 17,689 pages = 133 MBytes

Partitioning of an Ordered Domain

- The search structure (non-leaf part) of B+ tree forms a *partitioning* of an *ordered* domain (e.g., integer, string)



- Other indexes are unordered (even if a total order exists)
- Domains with >=2 dimensions: partial order only

Tree-structured indexes

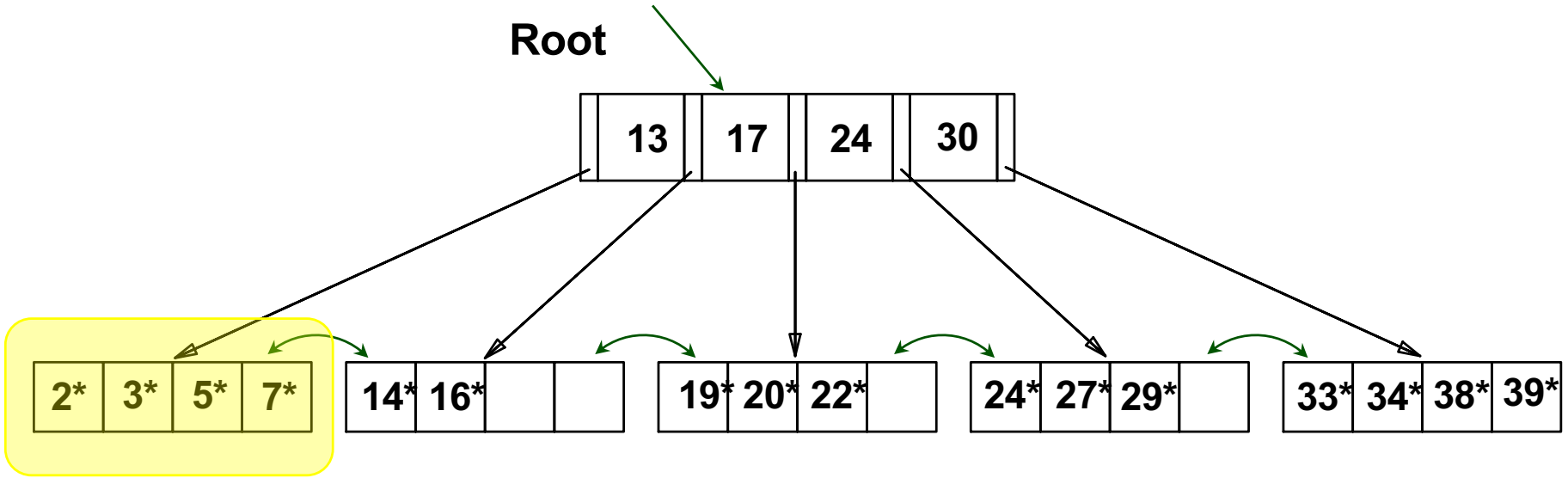
- Index concepts
- ISAM index: static structure
- B+ Tree index: dynamic balanced structure
  - Dynamic construction
  - Bulkloading

Inserting a Data Entry into a B+ Tree

- Find correct leaf *L* via a top-down search.
- Put data entry onto *L*.
  - If *L* has enough space, *done*
  - Else, must *split* *L (into L and a new node L2)*
    - Redistribute entries evenly, ***copy up*** middle key *k*, insert (*k*, pointer to *L2)* into parent of *L*.
  - Splitting can happen recursively to non-leaf nodes
    - Redistribute entries evenly, but ***push up*** middle key. (Contrast with leaf splits.)
- Splits "grow" the tree
  - First *wider*, then *one level taller* when the root splits.

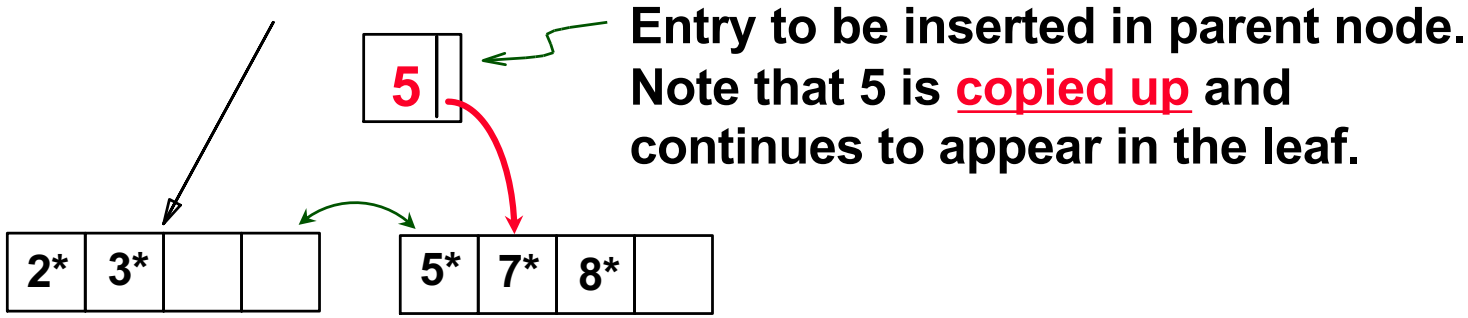
Previous Example

Inserting 8*



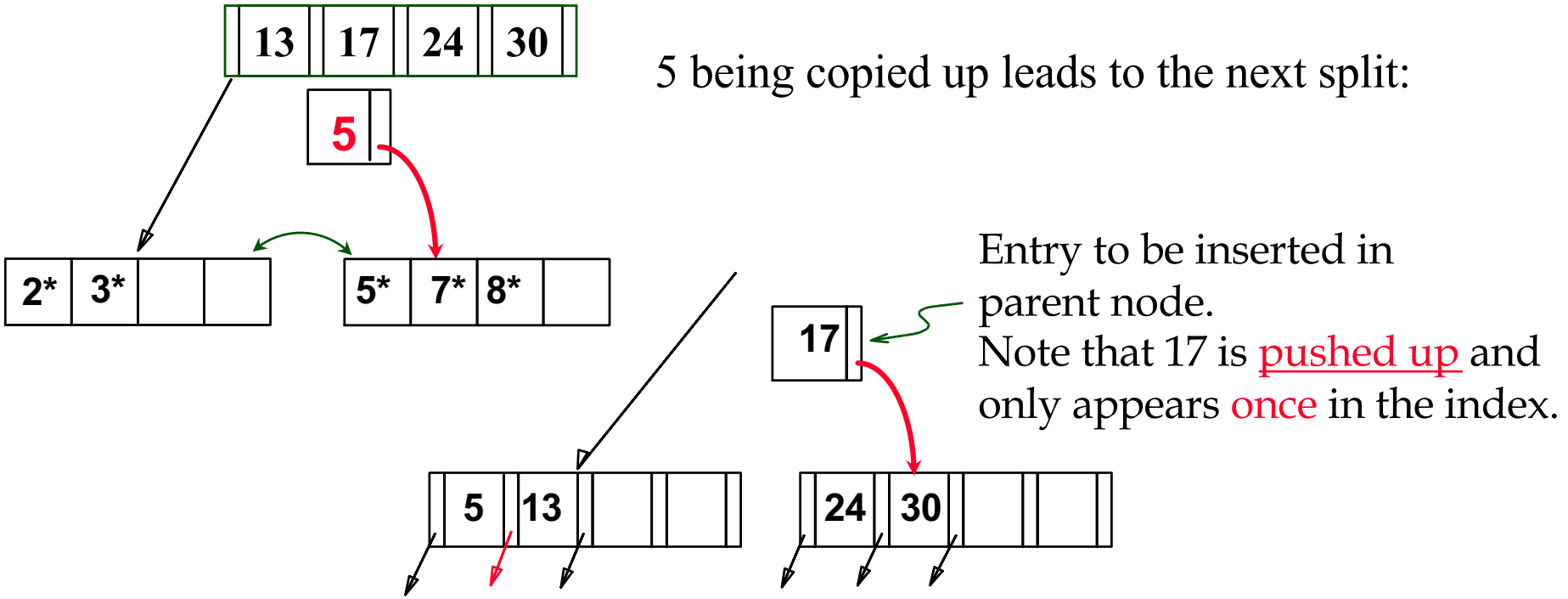
Node to be split next

Inserting 8 into Example B+ Tree*



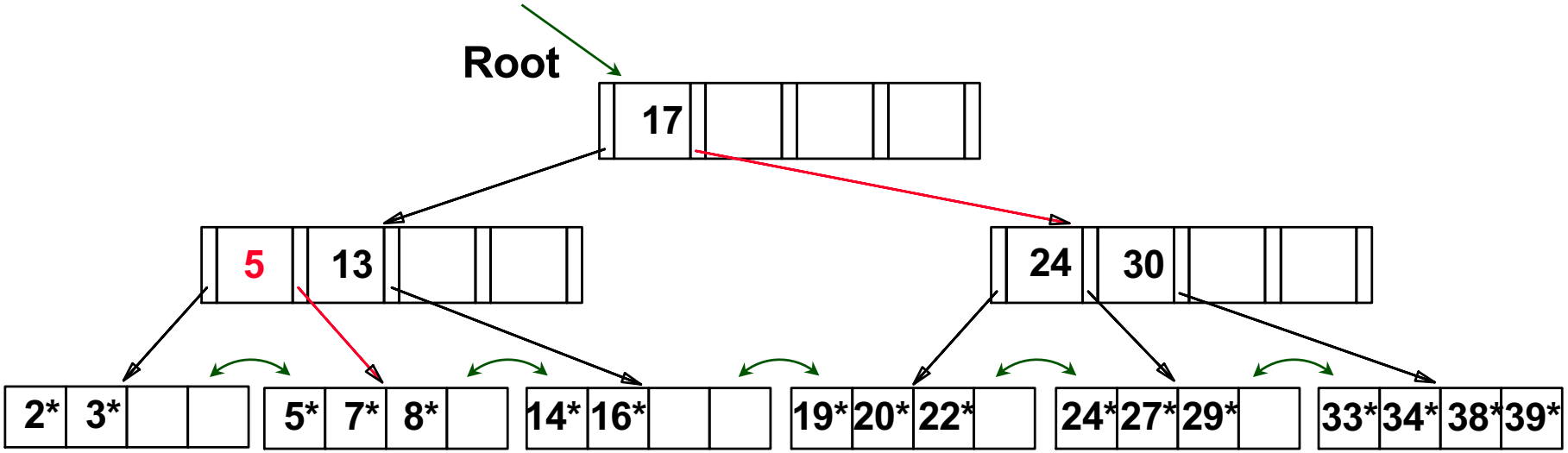
- Minimum occupancy is guaranteed in node splits.
- Copy up: key value of an inserted entry **must appear in a leaf node**

Inserting 8 into Example B+ Tree*



- Note difference between *copy-up* and *push-up*
  - **Copy-up**: occurs *when splitting a leaf*; the key entry remains in the leaf node, however the key is pushed up as a range delimiter in the parent
  - **Push-up**: occurs *when splitting an internal node*; the key serves as a range delimiter and it just moves up

*Example B+ Tree After Inserting 8**

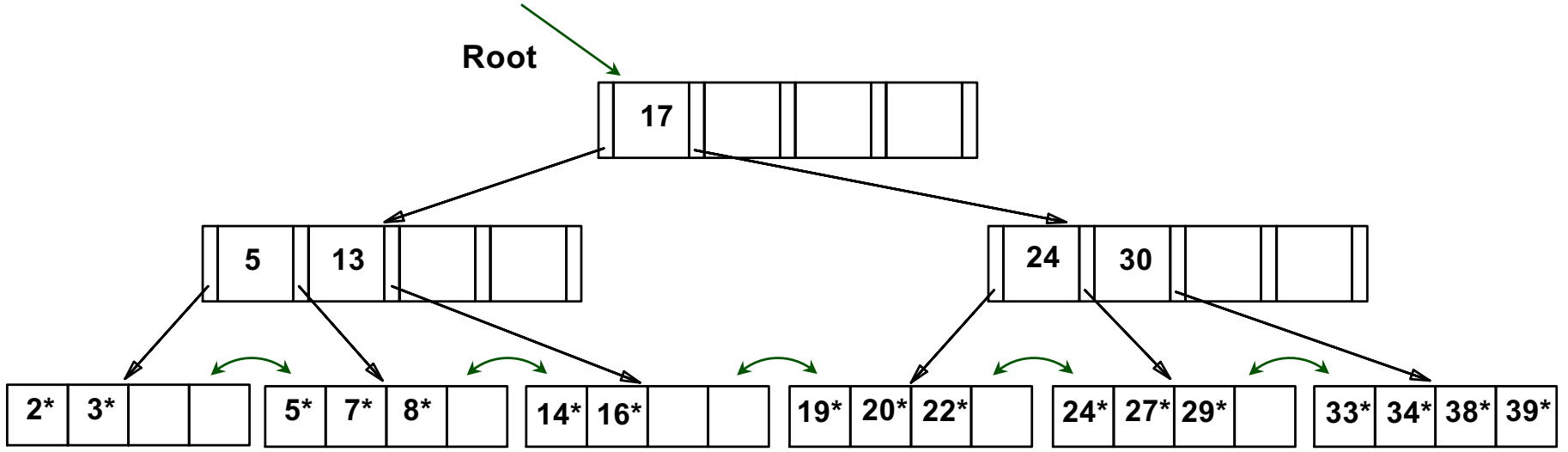


- Root was split, leading to increase in height

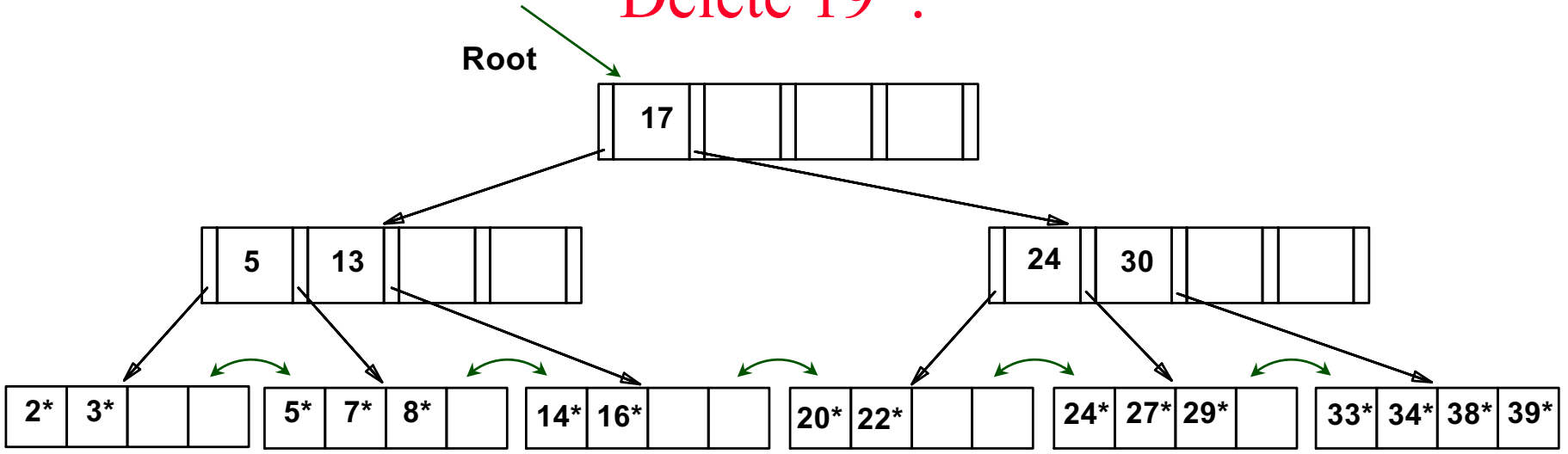
Deleting a Data Entry from a B+ Tree

- Start at root, find leaf *L* where entry belongs.
- Remove the entry.
  - If L is at least half-full, *done*
  - If L has only $\lfloor n/2 \rfloor - 1$ entries,
    - Try to *re-distribute*, borrowing from *sibling* (*adjacent node with same parent as L*).
    - If re-distribution fails, *merge* *L* and sibling. Must delete index entry (pointing to *L* or sibling) from parent of *L*.
- Merge could propagate to root, decreasing height.

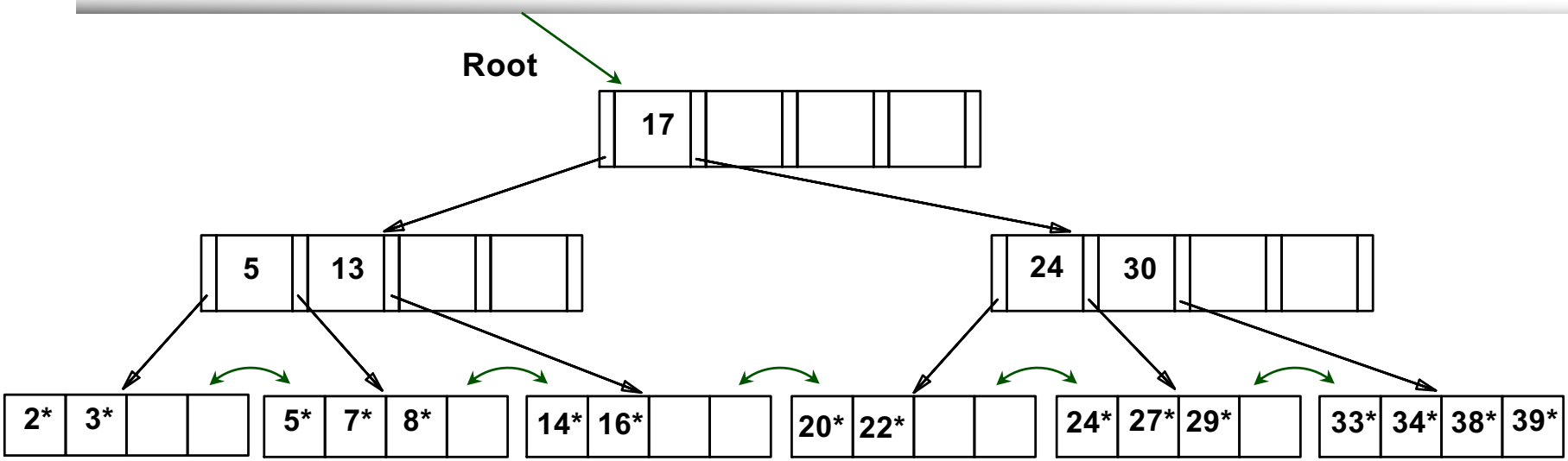
Deletion from B+ Tree



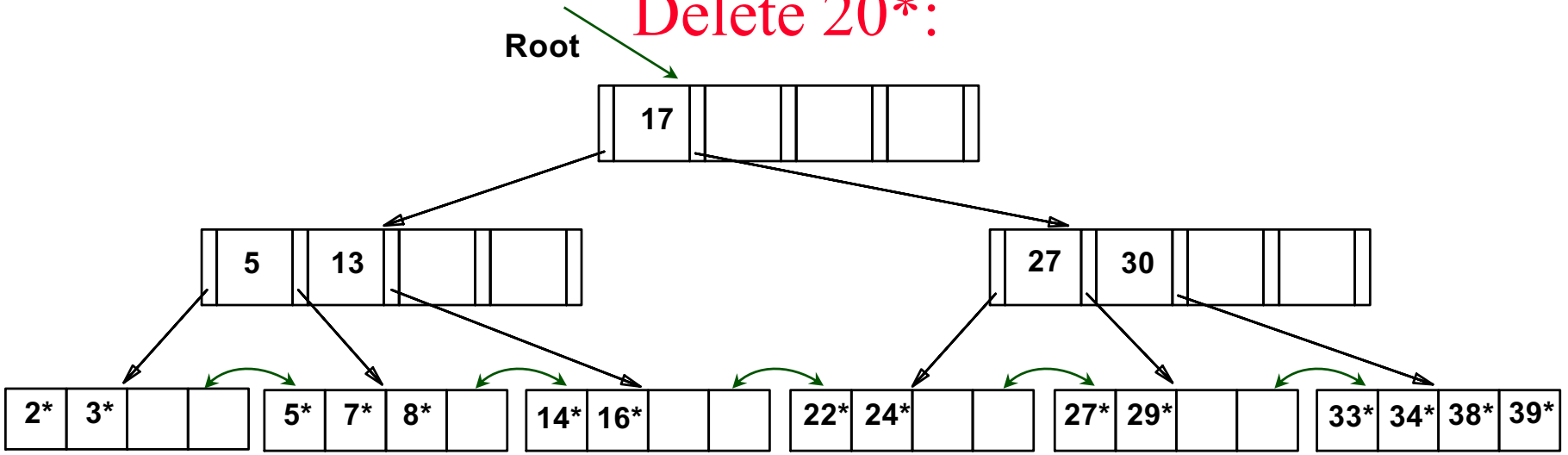
Delete 19*:



Deletion from B+ Tree

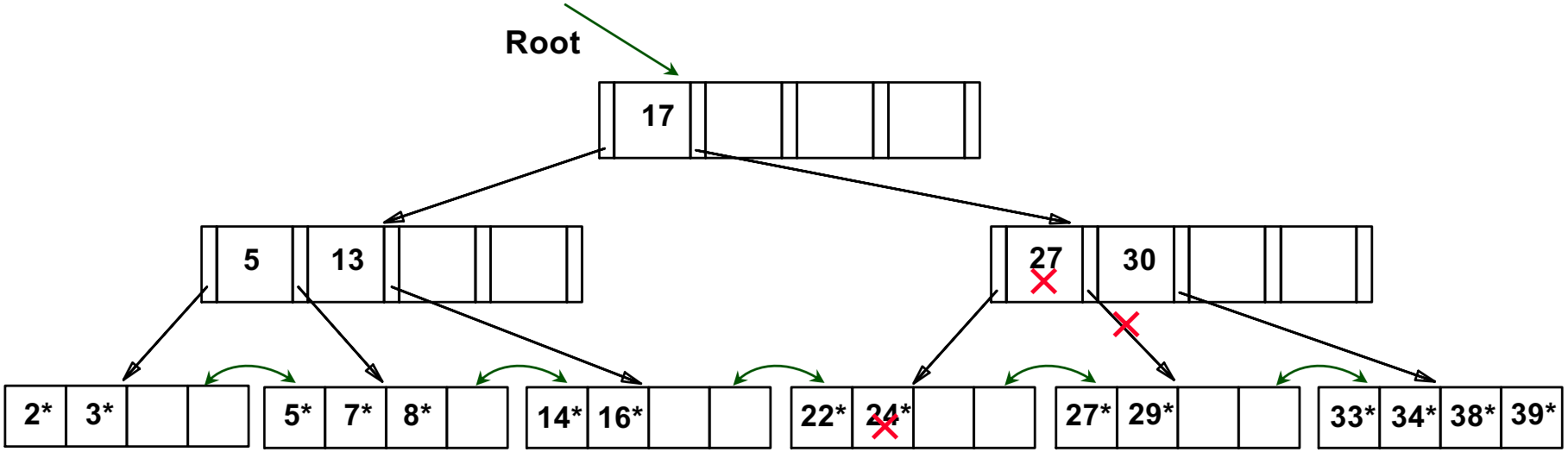


Delete 20*:



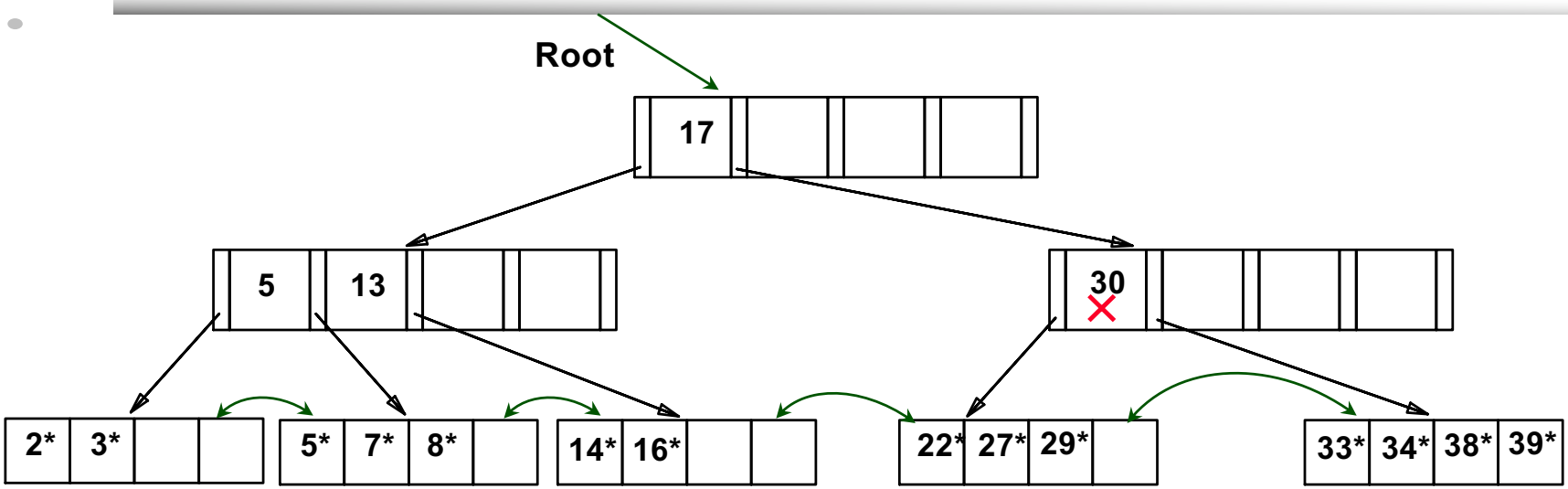
- Re-distribution. Notice how the middle key 27 is *copied up*.

Deleting 24 from the resulting B+ tree*

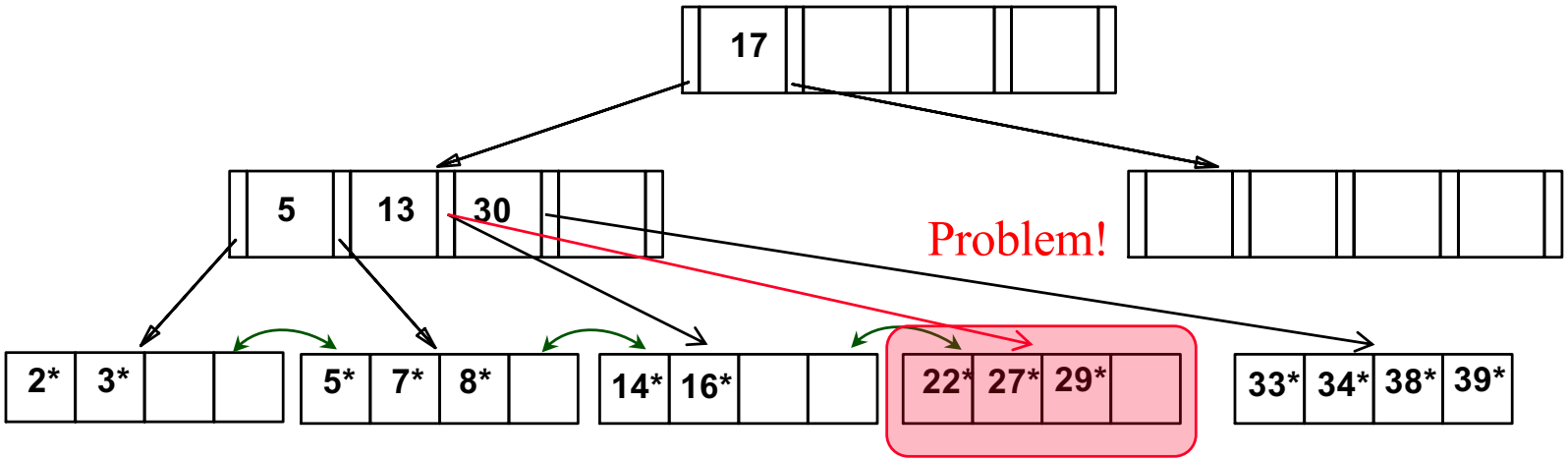


- Cannot redistribute: too few values in the sibling nodes
- Merge two children into one, adjust parent node
- Index entry **27** (which used to separate the merged children) is tossed (discarded)

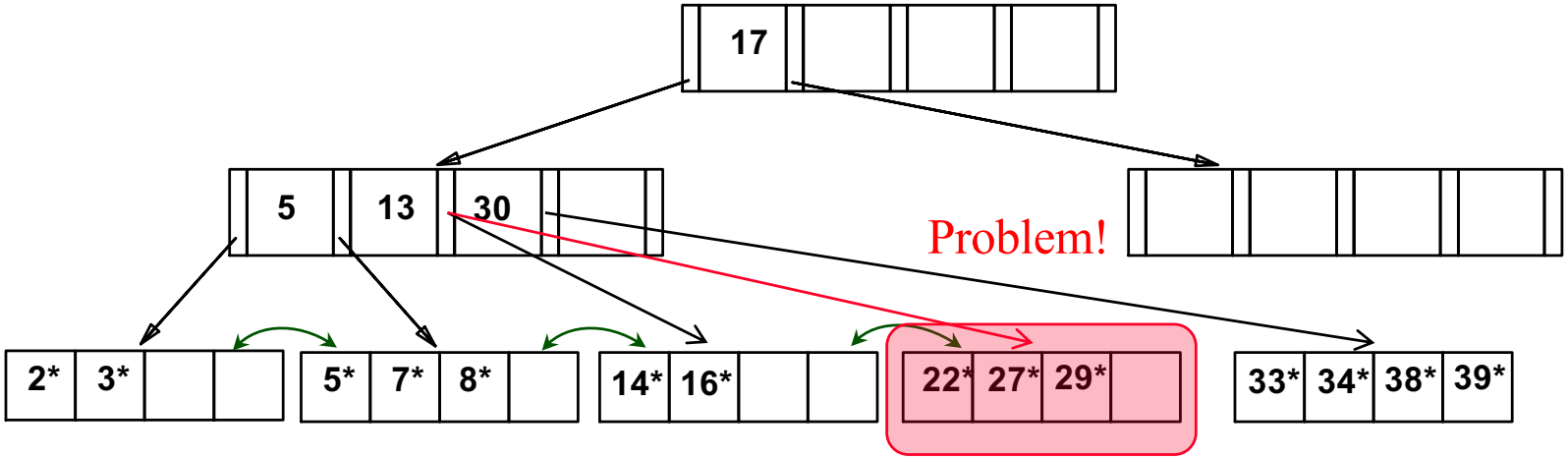
*Deleting 24**



- Intermediary state (unstable): the internal node containing 30 is too empty; we need to merge it with the left sibling

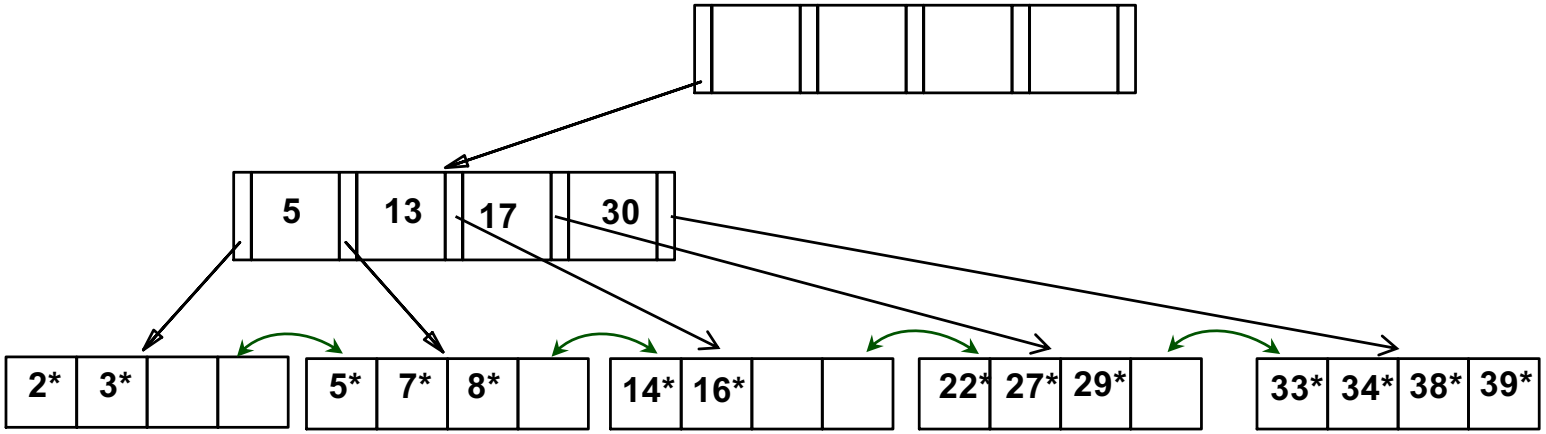


How to solve the problem?

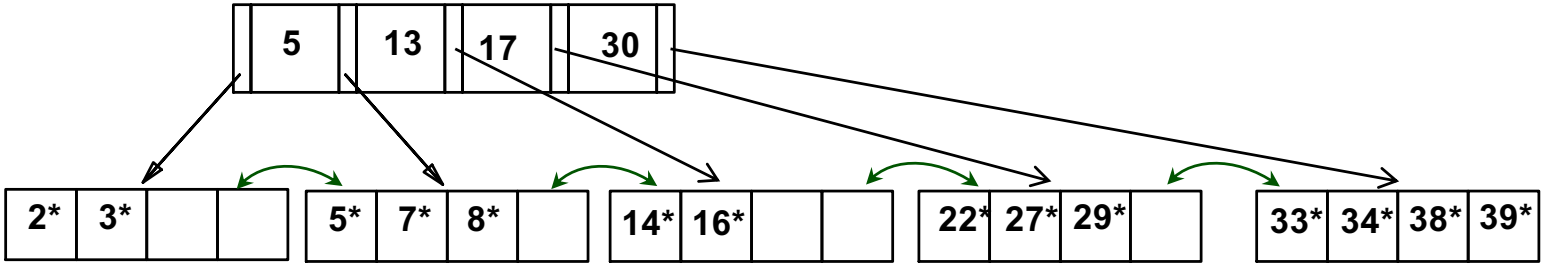


- The pointer at right of 13 represented "smaller than 17, larger than 13"
- The pointer at left of 30 represented "larger than 17, smaller than 30"
- 17 is exactly the delimiter needed to separate these
- **Solution**: pull down index key 17 to separate children of the new node

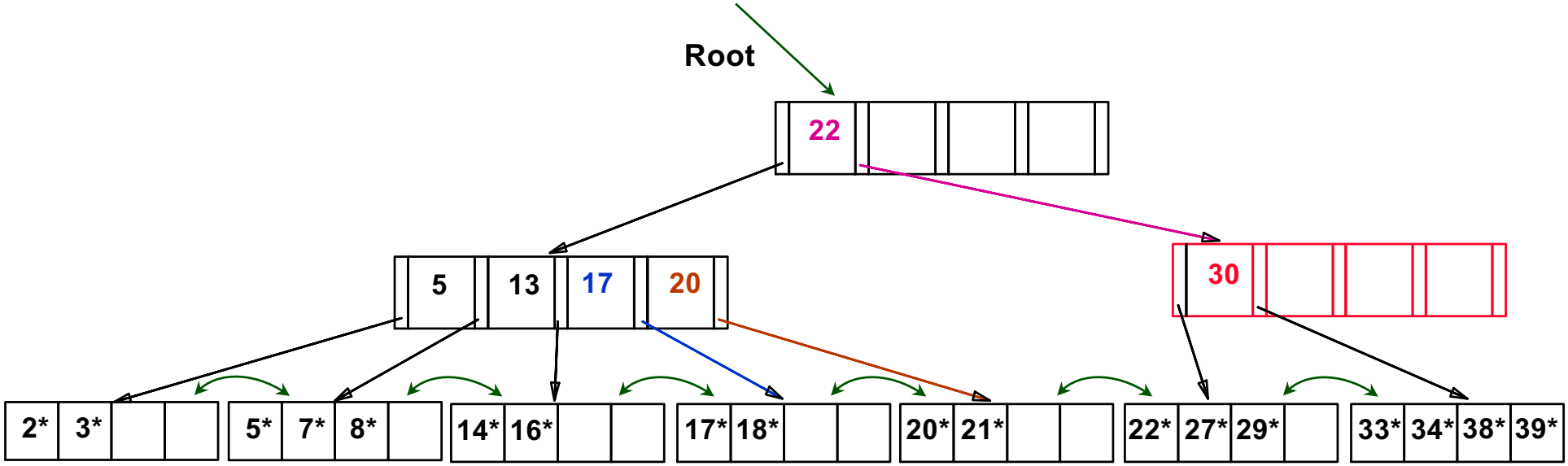
How to solve the problem?



- Deletion finished; root is empty → remove it

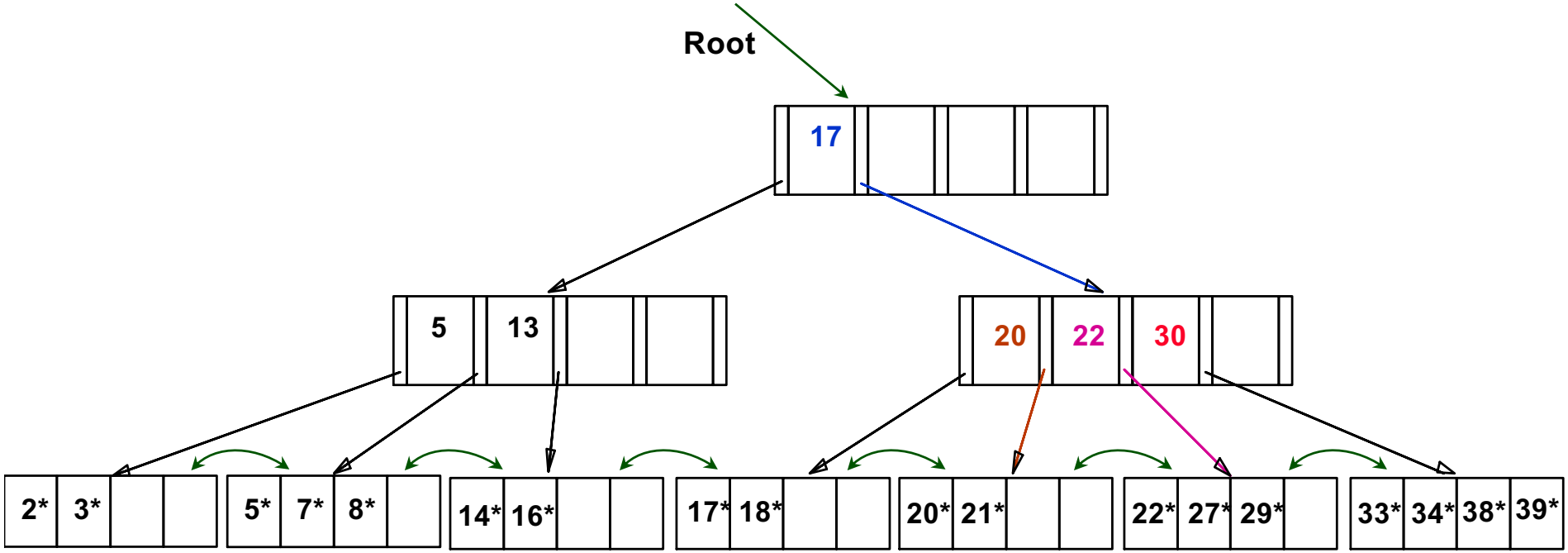


Another example: non-leaf redistribution



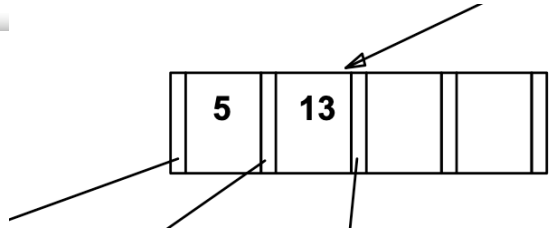
- Tree is shown *during deletion* of 24*. (What could be a possible initial tree?)
- In contrast to previous example, can re-distribute entry from left child of root to right child.

After re-distribution



- Intuitively, entries are re-distributed by '*pushing through*' the parent node.
- Re-distributing the entry with key 20 was minimally sufficient; we re-distributed 17 as well for illustration.

Prefix key compression



- Important to **increase fan-out**. (Why?)
- Key values in index entries only 'direct traffic'; can often compress (shorten) them.
  - E.g., adjacent index entries with search key values [*Dave Jones, David Smith, Devarakonda Murthy*]
  - Can compress *David Smith* to *Davi*.
  - In general, while compressing, must leave **each index entry**
    1. greater than **every key value (in any subtree) to its left**
    2. smaller than **every key value in the subtree to its right.**
- Insert/delete must be suitably modified.
- Supported in many commercial implementations

Tree-structured indexes

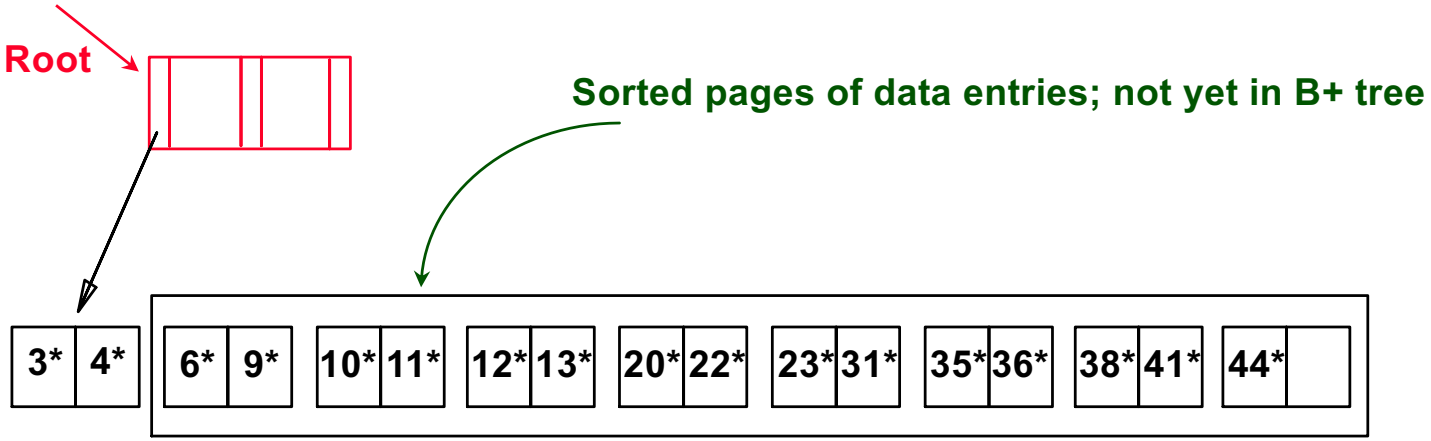
- Index concepts
- ISAM index: static structure
- B+ Tree index: dynamic balanced structure
  - Dynamic construction
  - Bulkloading

Bulk Loading of a B+ Tree

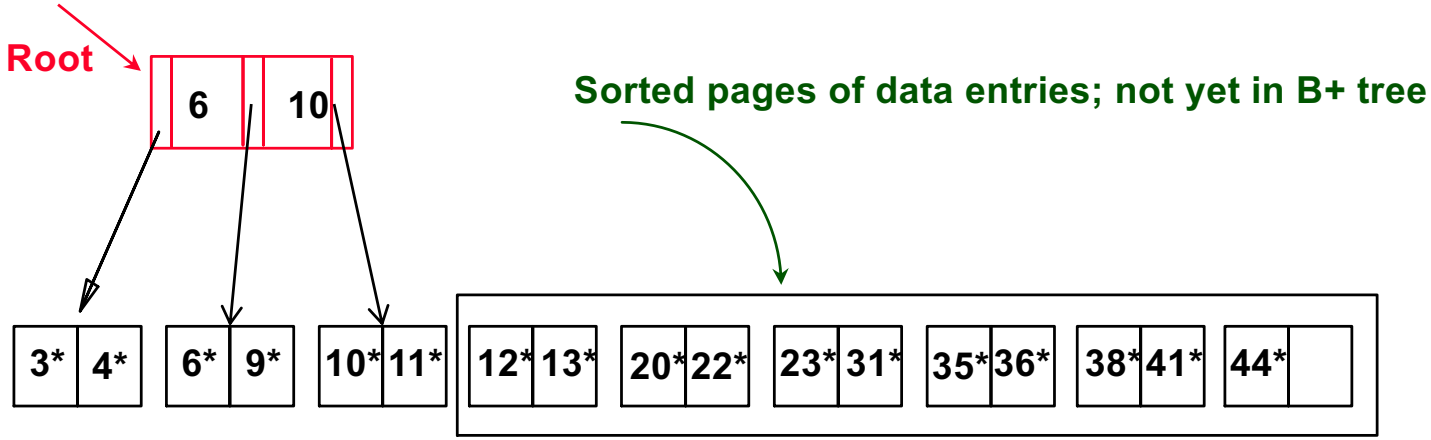
- Already have a large collection of records. Want to create a B+ tree index.
- **Option 1**: repeatedly inserting records
  - **Slow** due to repeated traversals and splits.
  - Not necessarily the optimal structure
  - Leaf pages may end up not full
    - 100% is not our goal, since every insert would mean a split
    - But all leaves at 50%, while correct, is not efficient
- **Option 2**: bulk loading (see next), more efficient

Bulk Loading Algorithm

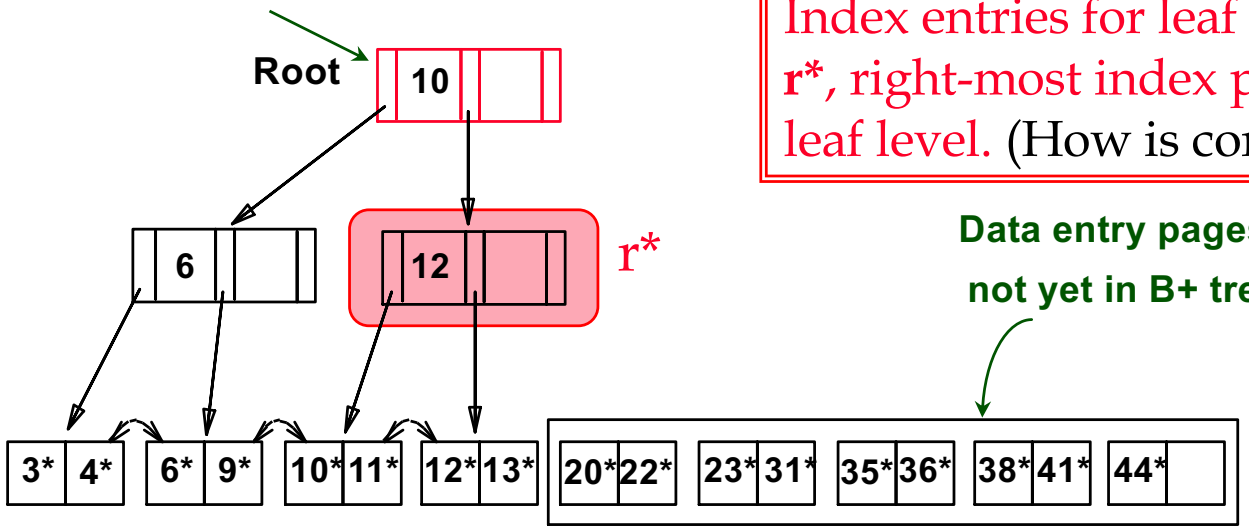
- *Initialization*:
  - Sort all data entries (we will learn external sorting soon…)
  - Insert pointer to the first (leaf) page in a new (root) page.



Bulk Loading Algorithm



- When the root is full, it splits. From this point on:

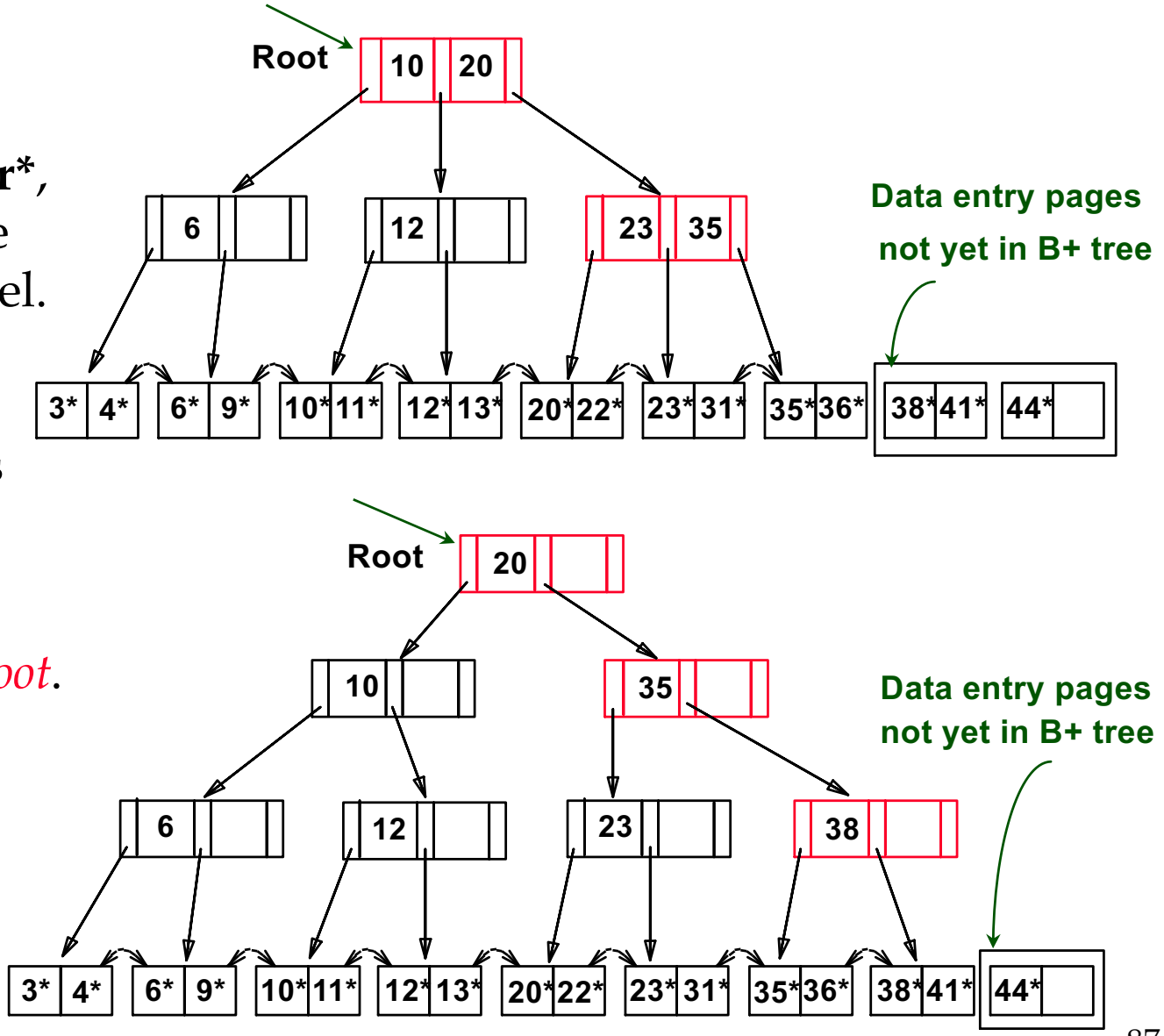


Index entries for leaf pages always go into **r***, right-most index page just above the leaf level. (How is correctness ensured?)

Data entry pages not yet in B+ tree

Bulk Loading Algorithm (Contd.)

- Index entries for leaf pages always go into **r***, right-most index page just above the leaf level.
- When the r* node fills up, it splits.
- Split may go up the *right-most path to the root*.



Questions

